



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

LAB EXPERIMENTS-CV

ITA0509-Computer Vision

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.TECH [AI&ML]

Submitted by

DASARI HARITHA(192425254)

Under the Supervision of

DR. YUVARAJ

SEP 2026

LAB EXPERIMENTS-CV

EXPERIMENT-1

AIM:

Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale.

PROGRAM:

```
import cv2

img = cv2.imread("flower.jpg")

if img is None:
    print("Image not found!")
else:
    # Show original image
    cv2.imshow("Original Image", img)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Gray Image", gray)

    # Save grayscale image
    cv2.imwrite("gray_flower.jpg", gray)

    print("Image converted successfully!")

    cv2.waitKey(0)

    cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-2

AIM:

Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur

PROGRAM:

```
import cv2

# Read the image

img = cv2.imread("flower.jpg")

# Check whether the image is loaded

if img is None:

    print("Image not found!")

else:

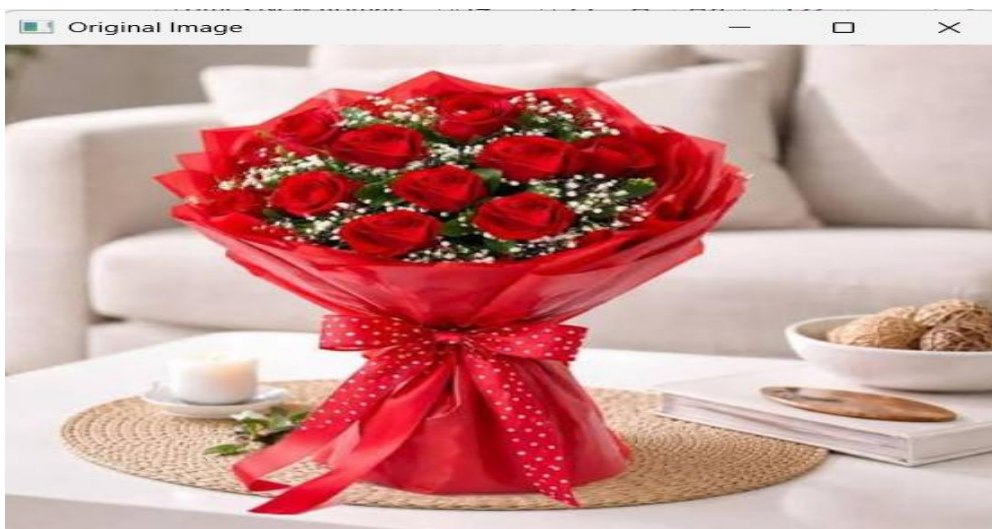
    # Display the original image

    cv2.imshow("Original Image", img)

    # Apply Gaussian Blur
```

```
blur = cv2.GaussianBlur(img, (15, 15), 0)
# Display the blurred image
cv2.imshow("Gaussian Blur Image", blur)
# Save the blurred image
cv2.imwrite("blur_flower.jpg", blur)
print("Gaussian Blur applied successfully!")
print("Blurred image saved as blur_flower.jpg")
# Wait until a key is pressed
cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-3

AIM:

Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide the Tkinter window
Tk().withdraw()

# Open a file dialog to choose an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if a file was selected
if not file_path:
    print("No image selected!")
    exit()

# Read the image
img = cv2.imread(file_path)

# Check if the image was loaded
if img is None:
    print("Error: Unable to load the image!")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply Canny Edge Detection
edges = cv2.Canny(gray, 100, 200)
```

```
# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Canny Edge Detection", edges)
# Wait until a key is pressed
cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-4

AIM:

Implement histogram equalization on the given image and compare it with the original image using Open CV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if image selected
if not file_path:
    print("No image selected!")
    exit()

# Read image
img = cv2.imread(file_path)

if img is None:
    print("Error: Unable to load image!")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Histogram Equalization
equalized = cv2.equalizeHist(gray)

# Display images
cv2.imshow("Original Grayscale Image", gray)
```

```
cv2.imshow("Histogram Equalized Image", equalized)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-5

AIM:

To analyze the histogram of the given input image based on color levels using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

from matplotlib import pyplot as plt

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if image selected
if not file_path:
    print("No image selected!")
    exit()

# Read image
img = cv2.imread(file_path)

if img is None:
    print("Error: Unable to load image!")
    exit()

# Display original image
cv2.imshow("Original Image", img)

# Plot histogram for B, G, R channels
colors = ('b', 'g', 'r')

plt.figure("Color Histogram")

for i, color in enumerate(colors):
```

```

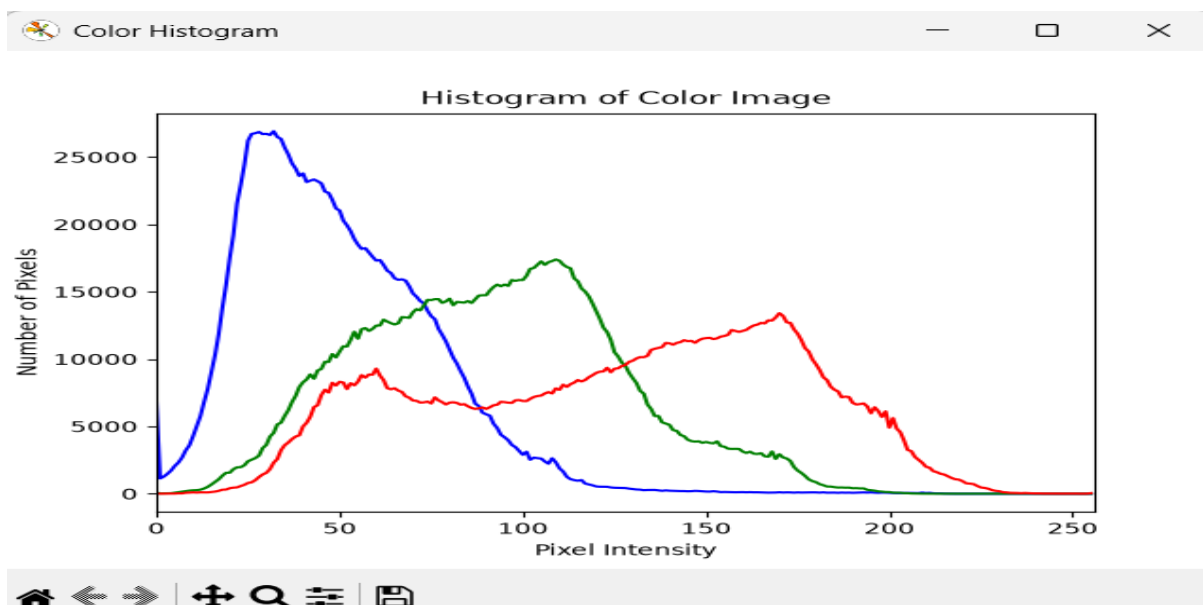
hist = cv2.calcHist([img], [i], None, [256], [0, 256])
plt.plot(hist, color=color)
plt.xlim([0, 256])
plt.title("Histogram of Color Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
plt.show()
cv2.waitKey(0)
cv2.destroyAllWindows()

```

INPUT:



OUTPUT:



EXPERIMENT-6

AIM:

To perform basic image handling and processing operations by reading an image in Python and applying the **Erode** function using OpenCV.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if image selected
if not file_path:
    print("No image selected!")
    exit()

# Read image
img = cv2.imread(file_path)

if img is None:
    print("Error: Unable to load image!")
    exit()

# Create a kernel
kernel = np.ones((5, 5), np.uint8)

# Apply erosion
eroded = cv2.erode(img, kernel, iterations=1)
```

```
# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Eroded Image", eroded)
# Wait until a key is pressed
cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-7

AIM:

To perform basic video processing by reading a captured video in Python and displaying it in **normal**, **slow motion**, and **fast motion** using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select video
video_path = filedialog.askopenfilename(
    title="Select a Video",
    filetypes=[("Video Files", "*.mp4 *.avi *.mov *.mkv")]
)

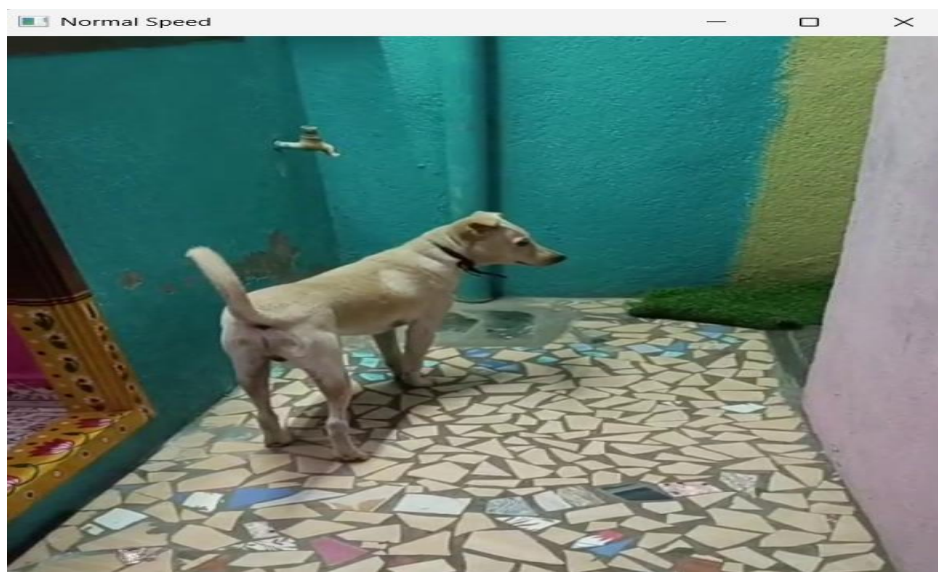
# Check if a video is selected
if not video_path:
    print("No video selected!")
    exit()

# ----- Normal Speed -----
cap = cv2.VideoCapture(video_path)

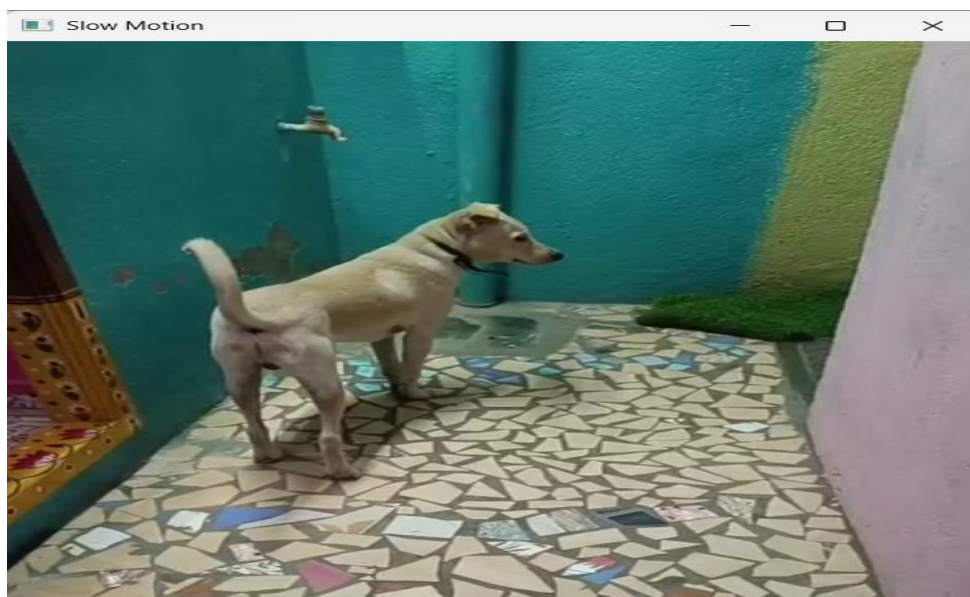
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Normal Speed", frame)
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

```
# ----- Slow Motion -----  
cap = cv2.VideoCapture(video_path)  
while cap.isOpened():  
    ret, frame = cap.read()  
    if not ret:  
        break  
    cv2.imshow("Slow Motion", frame)  
    if cv2.waitKey(100) & 0xFF == ord('q'):  
        break  
cap.release()  
cv2.destroyAllWindows()  
  
# ----- Fast Motion -----  
cap = cv2.VideoCapture(video_path)  
while cap.isOpened():  
    ret, frame = cap.read()  
    if not ret:  
        break  
    cv2.imshow("Fast Motion", frame)  
    if cv2.waitKey(5) & 0xFF == ord('q'):  
        break  
cap.release()  
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-8

AIM:

To perform basic image handling and processing operations by reading an image in Python and applying the **Dilate** function using OpenCV.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if an image was selected
if not file_path:
    print("No image selected!")
    exit()

# Read the image
img = cv2.imread(file_path)

# Check if image loaded successfully
if img is None:
    print("Error: Unable to load image!")
    exit()

# Create a 5x5 kernel
kernel = np.ones((5, 5), np.uint8)

# Apply Dilation
dilated = cv2.dilate(img, kernel, iterations=1)
```

```
# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Dilated Image", dilated)

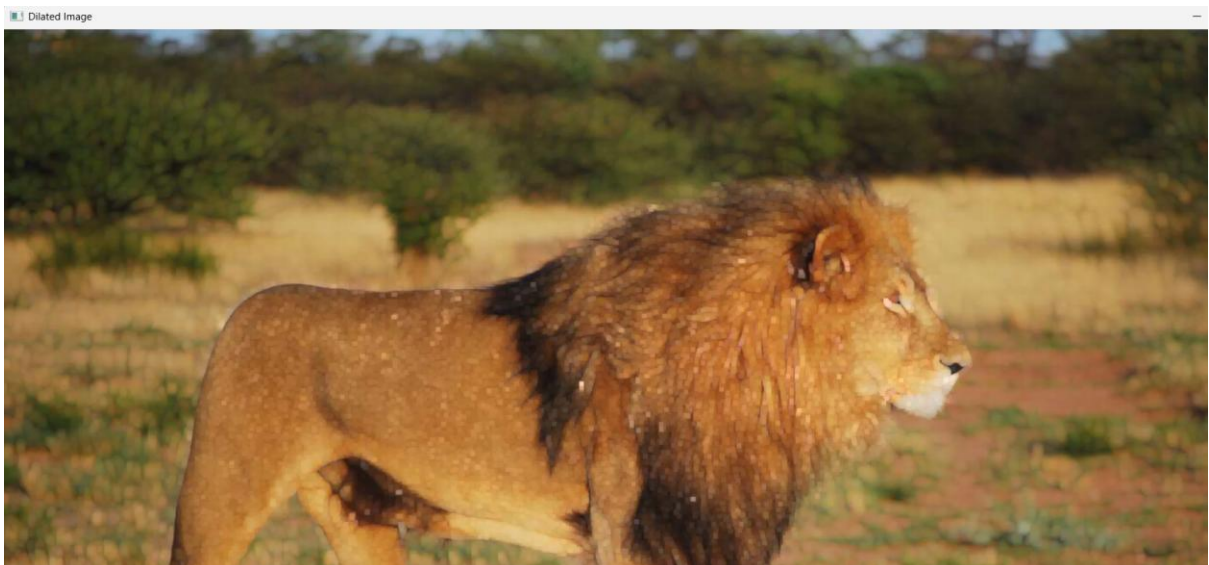
# Wait for a key press
cv2.waitKey(0)

# Close all windows
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-9

AIM:

To implement image scaling techniques to resize an image to both bigger and smaller sizes using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if an image was selected
if not file_path:
    print("No image selected!")
    exit()

# Read the image
img = cv2.imread(file_path)

# Check if image loaded successfully
if img is None:
    print("Error: Unable to load image!")
    exit()

# Resize image to bigger size (2 times)
bigger = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)

# Resize image to smaller size (half)
smaller = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
```

```
# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Bigger Image", bigger)
cv2.imshow("Smaller Image", smaller)

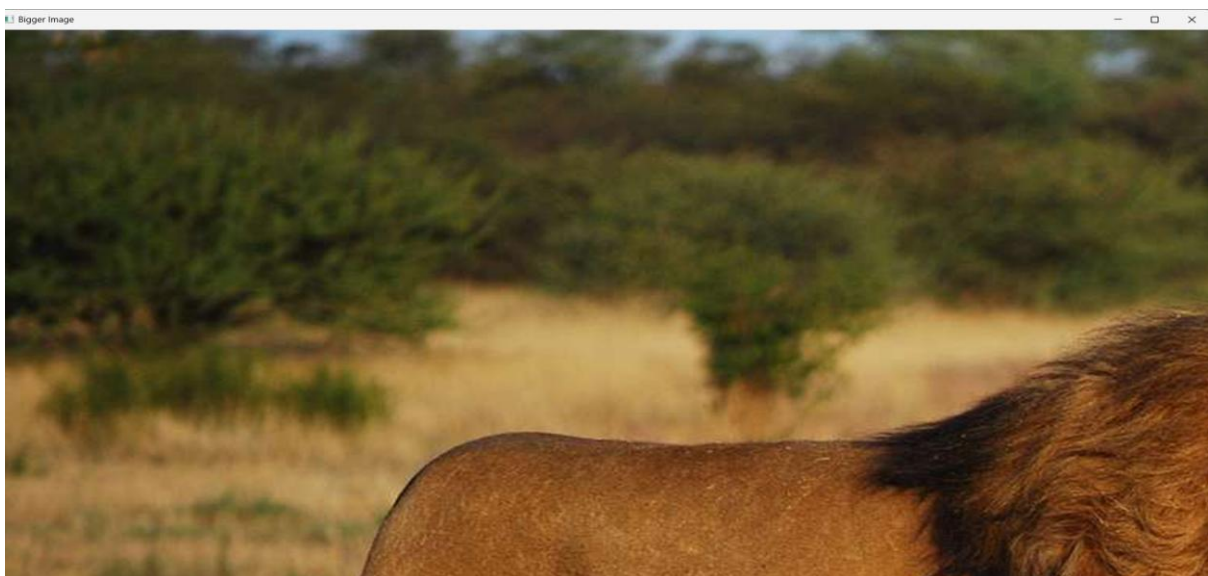
# Wait for a key press
cv2.waitKey(0)

# Close all windows
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-10

AIM:

To perform a **90-degree clockwise rotation** on the given image using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if an image was selected
if not file_path:
    print("No image selected!")
    exit()

# Read the image
img = cv2.imread(file_path)

# Check if image loaded successfully
if img is None:
    print("Error: Unable to load image!")
    exit()

# Rotate the image 90 degrees clockwise
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("90 Degree Clockwise Rotated Image", rotated)

# Wait for a key press
```

```
cv2.waitKey(0)
```

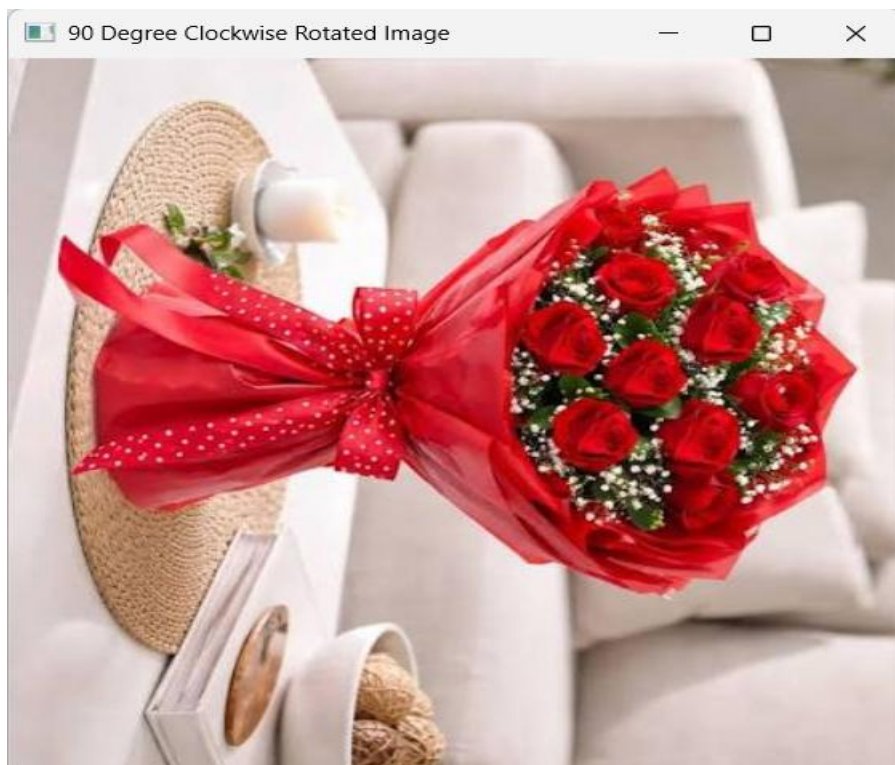
```
# Close all windows
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-11

AIM:

To perform a **180-degree clockwise rotation** on the given image using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if an image was selected
if not file_path:
    print("No image selected!")
    exit()

# Read the image
img = cv2.imread(file_path)

# Check if image loaded successfully
if img is None:
    print("Error: Unable to load image!")
    exit()

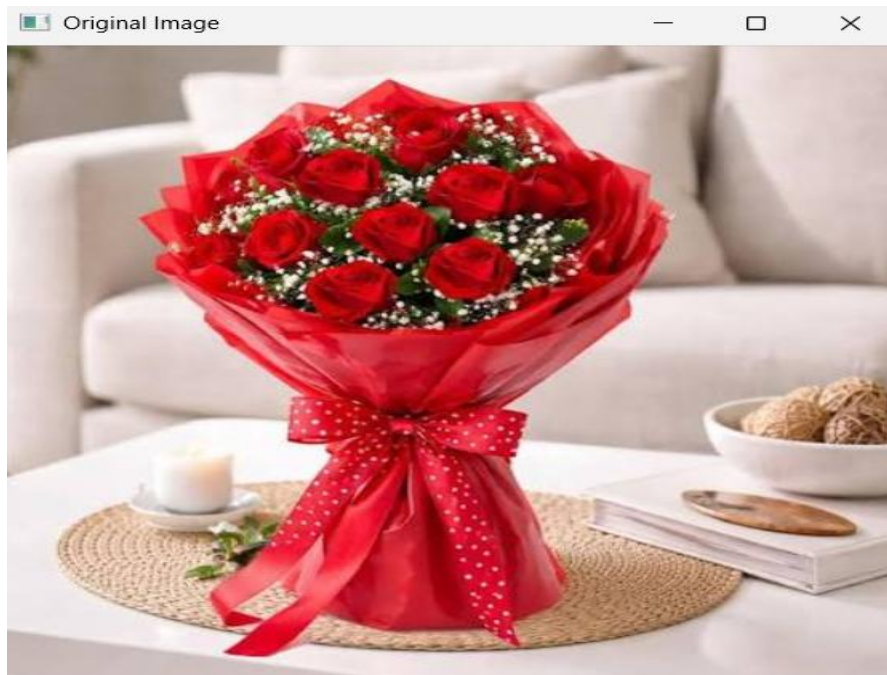
# Rotate the image 180 degrees
rotated = cv2.rotate(img, cv2.ROTATE_180)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("180 Degree Rotated Image", rotated)
```

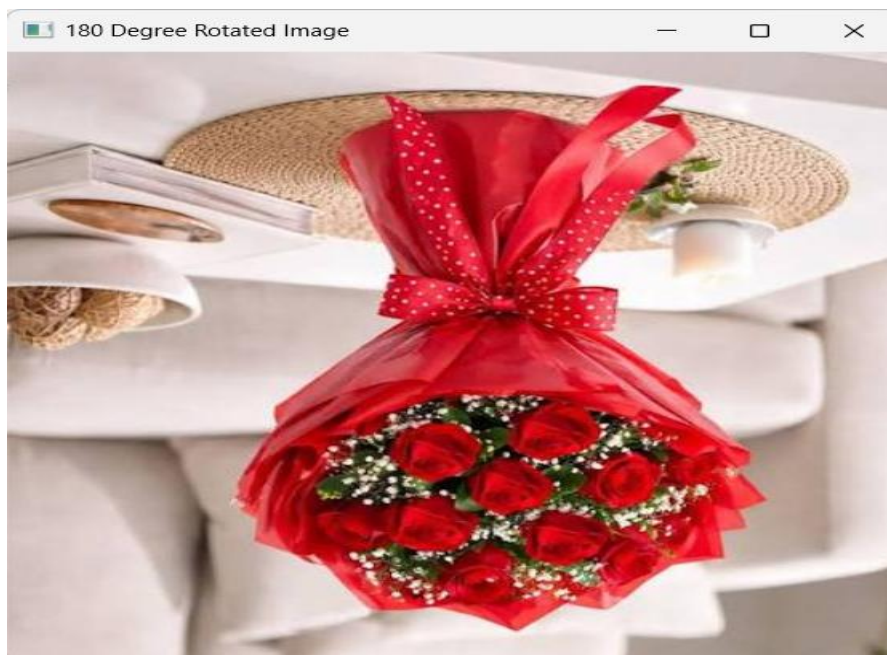
```
# Wait for a key press
cv2.waitKey(0)

# Close all windows
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-12

AIM:

To perform a 270° clockwise rotation on the given image using Python and OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

# Check if an image was selected
if not file_path:
    print("No image selected!")
    exit()

# Read the image
img = cv2.imread(file_path)

# Check if image loaded successfully
if img is None:
    print("Error: Unable to load image!")
    exit()

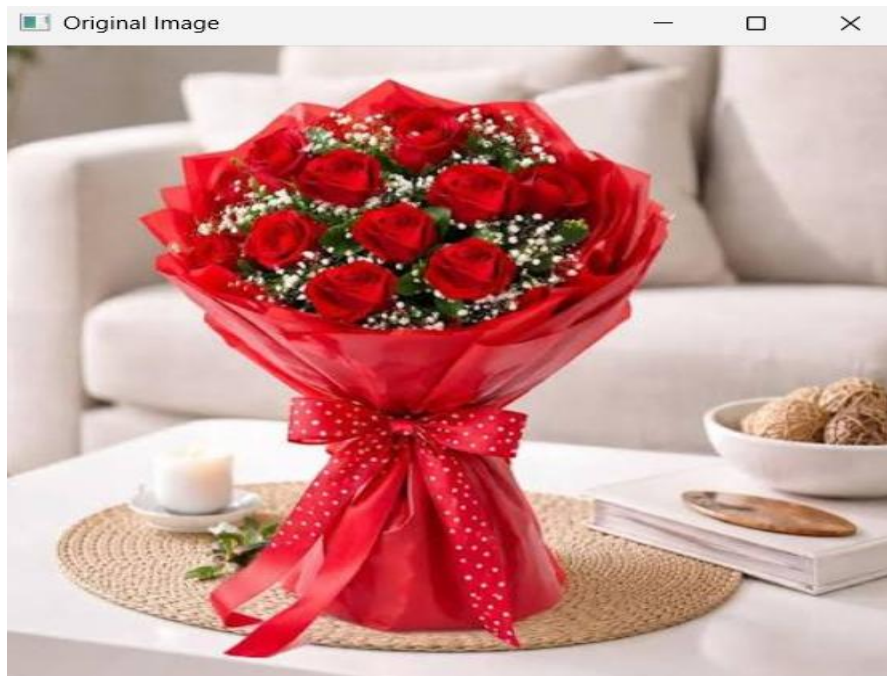
# Rotate the image 270 degrees clockwise
rotated = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("270 Degree Rotated Image", rotated)
```

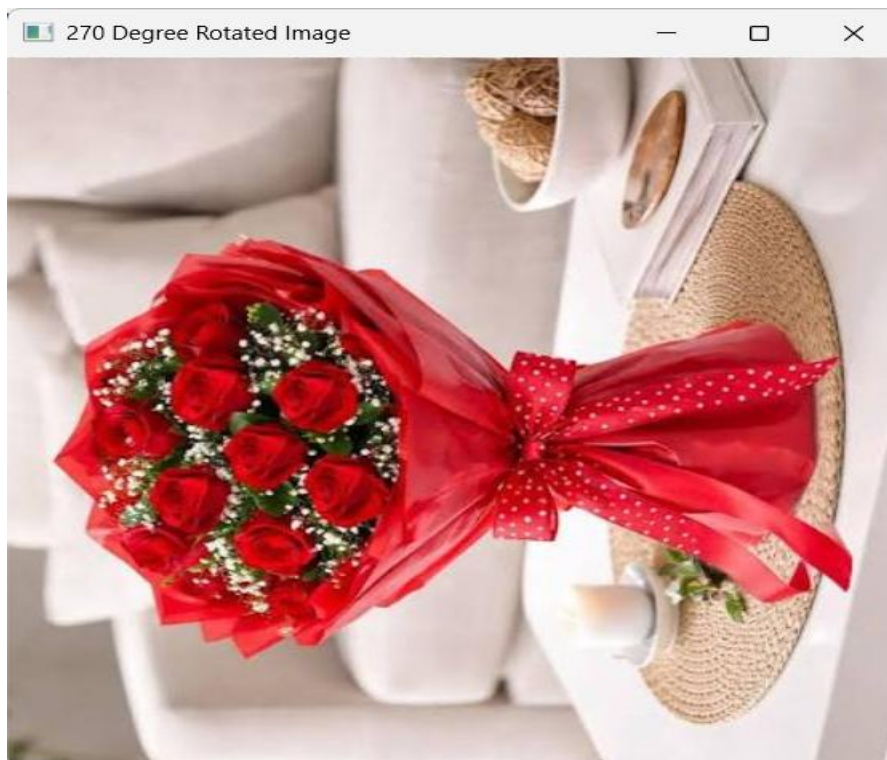
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-13

AIM:

To perform Affine Transformation using Python and OpenCV.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No image selected!")
    exit()

img = cv2.imread(file_path)

if img is None:
    print("Error: Unable to load image!")
    exit()

rows, cols = img.shape[:2]

pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])

matrix = cv2.getAffineTransform(pts1, pts2)

result = cv2.warpAffine(img, matrix, (cols, rows))

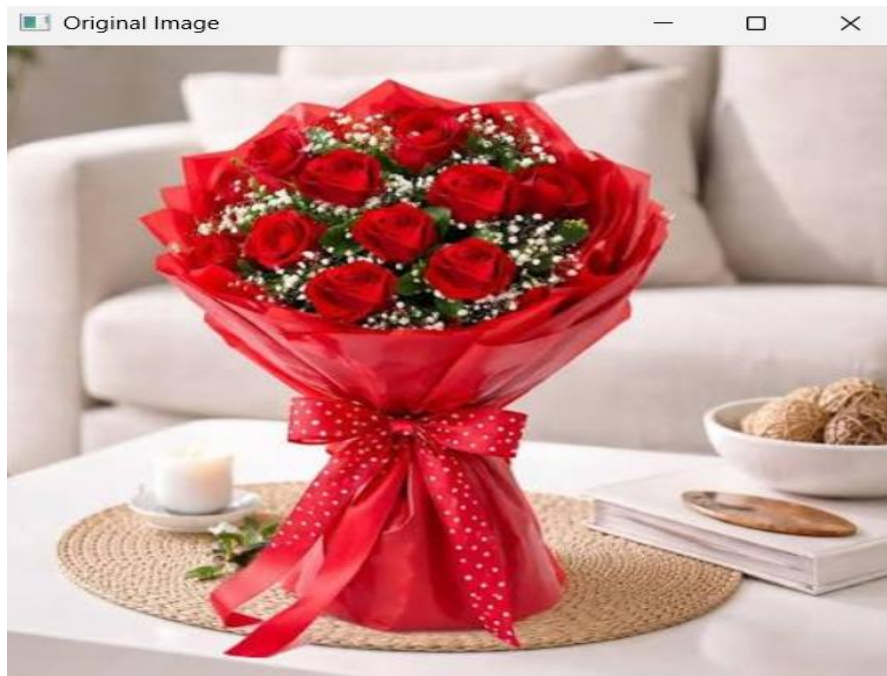
cv2.imshow("Original Image", img)

cv2.imshow("Affine Transformation", result)
```

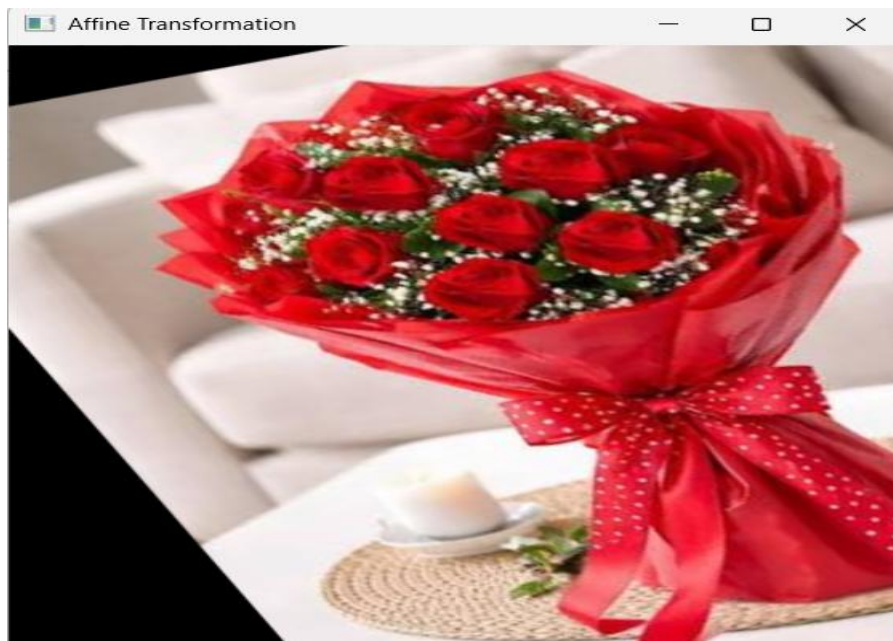
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-14

AIM:

To perform Perspective Transformation using Python and OpenCV.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No image selected!")
    exit()

img = cv2.imread(file_path)

if img is None:
    print("Error: Unable to load image!")
    exit()

rows, cols = img.shape[:2]

pts1 = np.float32([[50,50],[300,50],[50,300],[300,300]])
pts2 = np.float32([[0,0],[300,0],[100,300],[300,300]])

matrix = cv2.getPerspectiveTransform(pts1, pts2)

result = cv2.warpPerspective(img, matrix, (cols, rows))

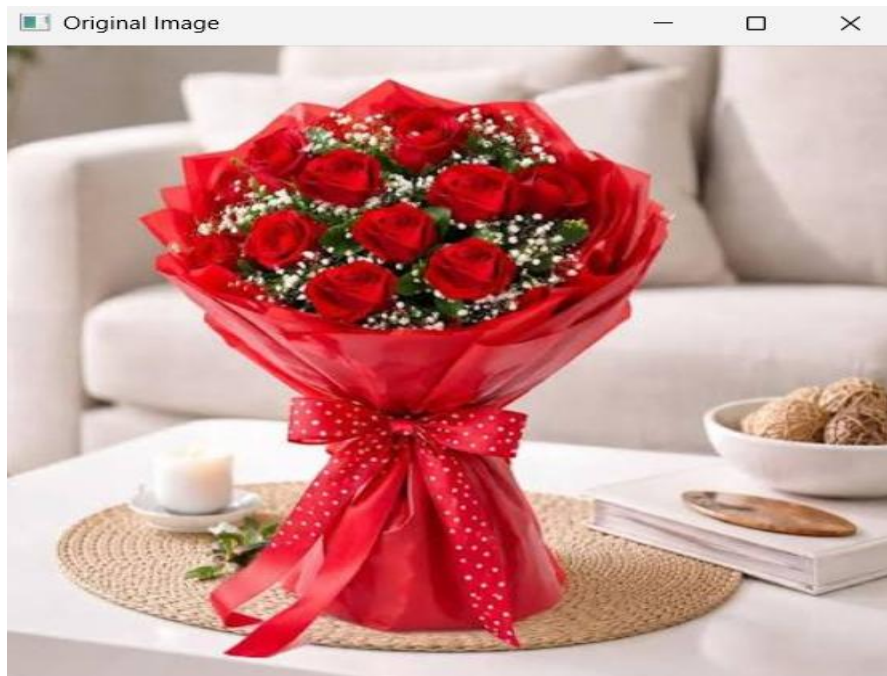
cv2.imshow("Original Image", img)

cv2.imshow("Perspective Transformation", result)
```

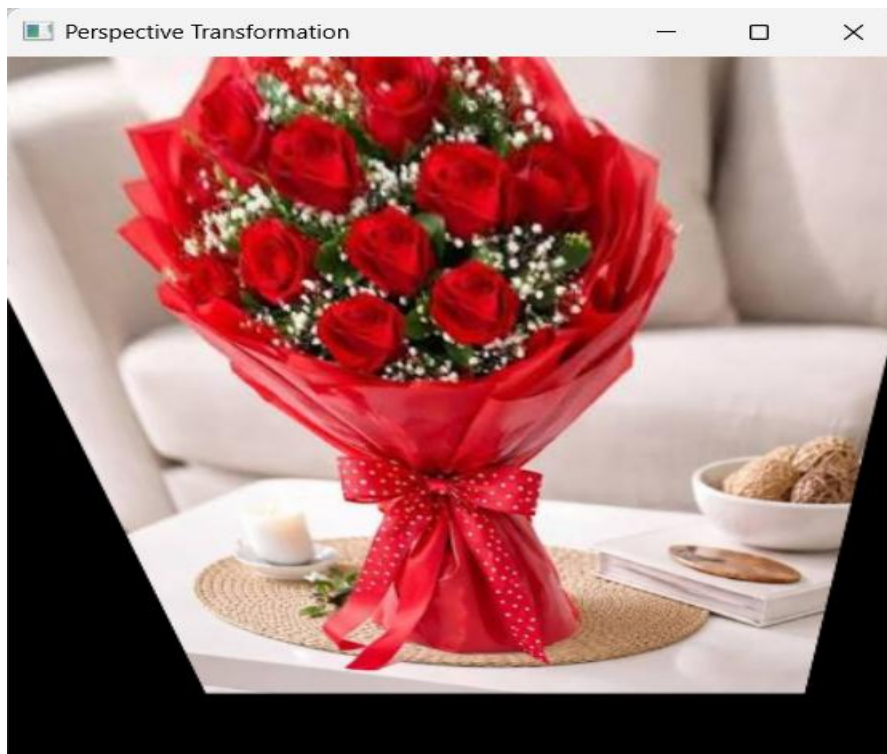
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-15

AIM:

To detect corners using the Harris Corner Detection method.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select an image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No image selected!")
    exit()

img = cv2.imread(file_path)

if img is None:
    print("Error: Unable to load image!")
    exit()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

corners = cv2.cornerHarris(gray, 2, 3, 0.04)
corners = cv2.dilate(corners, None)

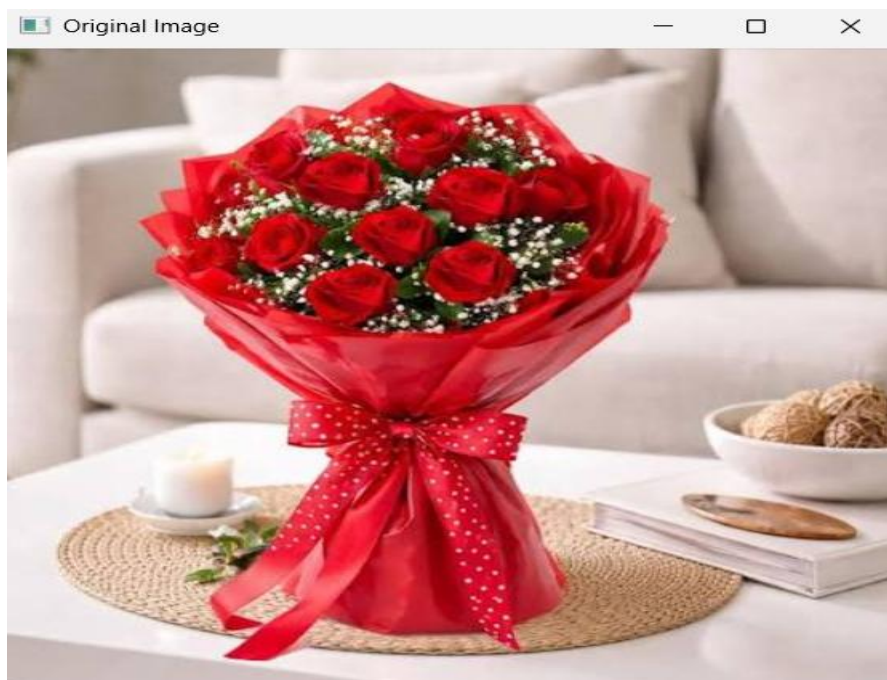
img[corners > 0.01 * corners.max()] = [0,0,255]

cv2.imshow("Harris Corner Detection", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-16

AIM:

To implement the **Sobel Edge Detection Algorithm** using OpenCV to detect the edges in an input image.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No image selected!")
    exit()

img = cv2.imread(file_path)

if img is None:
    print("Unable to load image!")
    exit()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel X
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

# Sobel Y
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

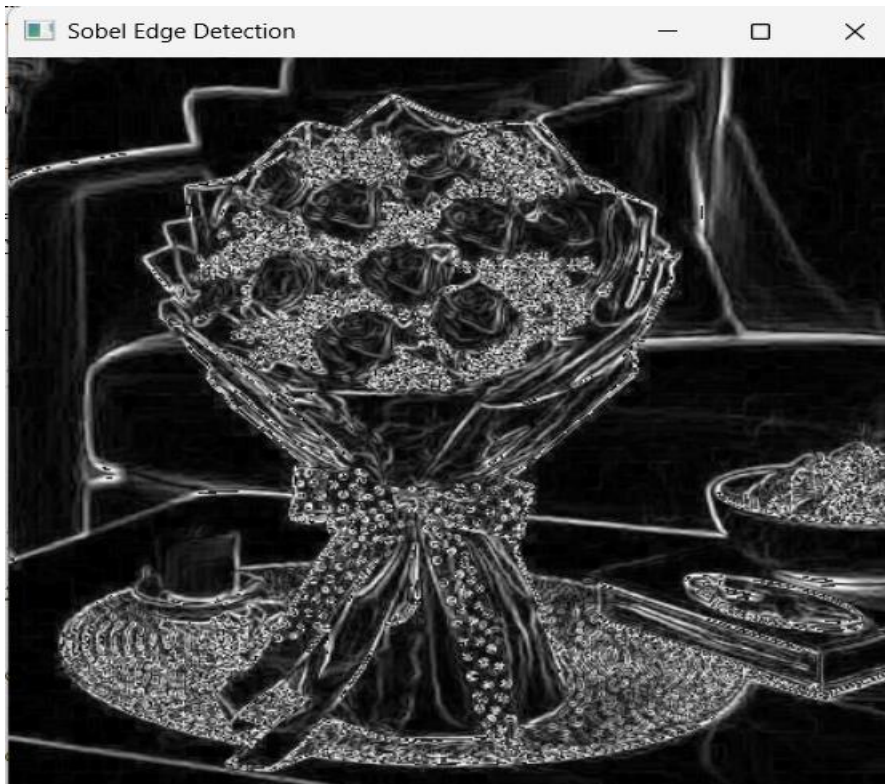
# Combine
sobel = cv2.magnitude(sobelx, sobely)
```

```
cv2.imshow("Original Image", img)
cv2.imshow("Sobel Edge Detection", sobel.astype(np.uint8))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-17

AIM:

To implement a watermarking technique by inserting text into an image using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No image selected!")
    exit()

img = cv2.imread(file_path)

if img is None:
    print("Unable to load image!")
    exit()

watermark = img.copy()

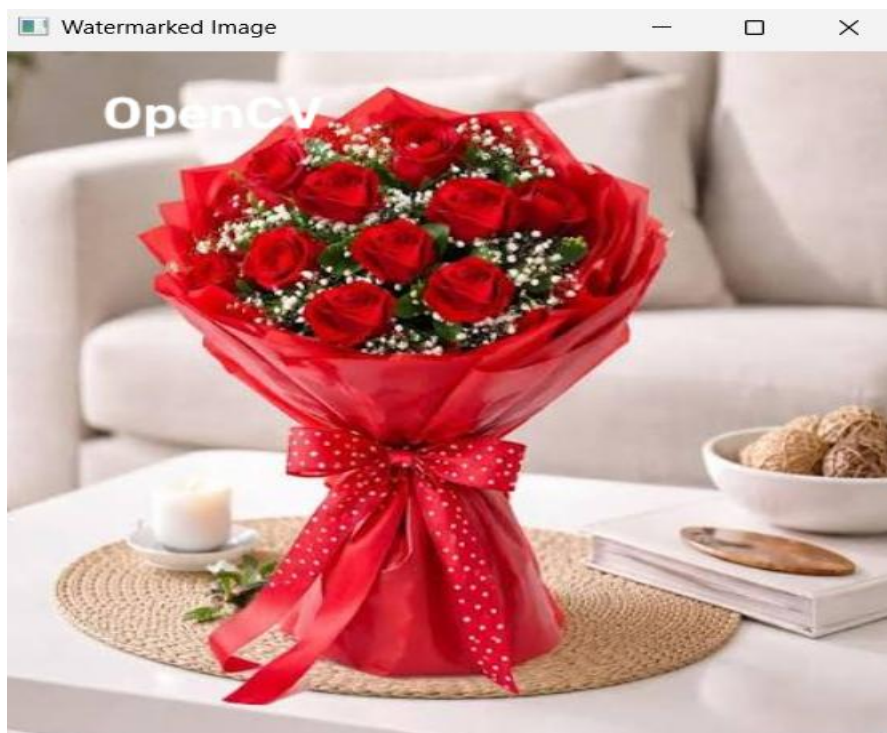
cv2.putText(
    watermark,
    "OpenCV",
    (50, 50),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (255, 255, 255),
    2
```

```
)  
cv2.imshow("Original Image", img)  
cv2.imshow("Watermarked Image", watermark)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-18

AIM:

To implement image cropping, copying, and pasting using the Region of Interest (ROI) concept in OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No image selected!")
    exit()

img = cv2.imread(file_path)

if img is None:
    print("Unable to load image!")
    exit()

# Crop ROI
roi = img[50:200, 50:200]

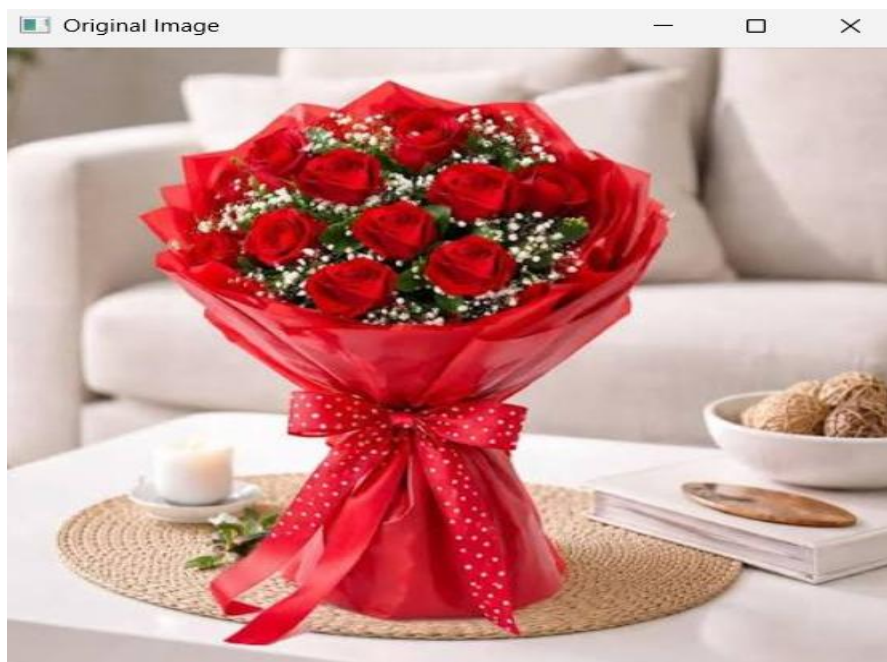
# Paste ROI
img[220:370, 220:370] = roi

cv2.imshow("ROI Cropping, Copying and Pasting", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-19

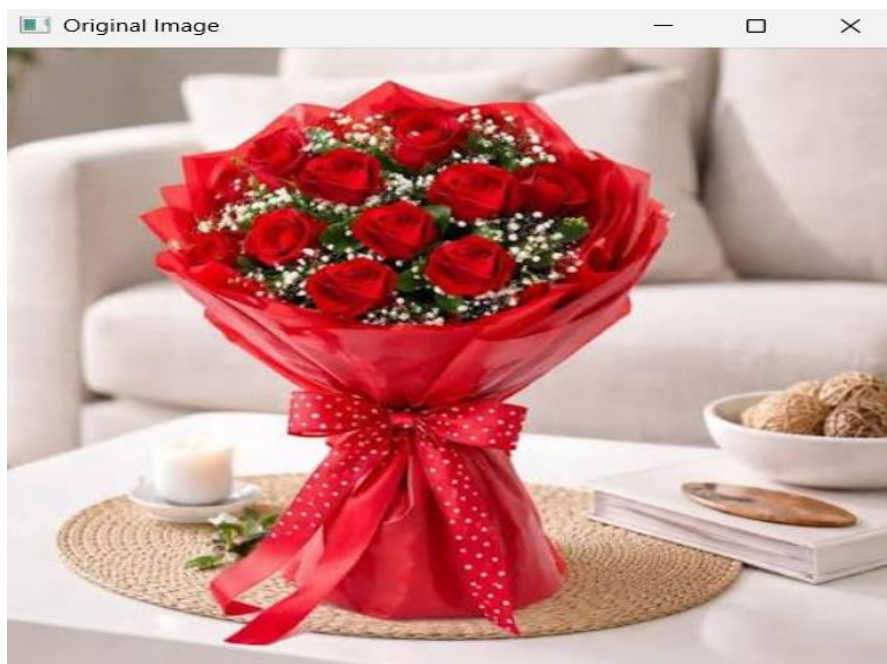
AIM:

To implement the erosion morphological operation using OpenCV.

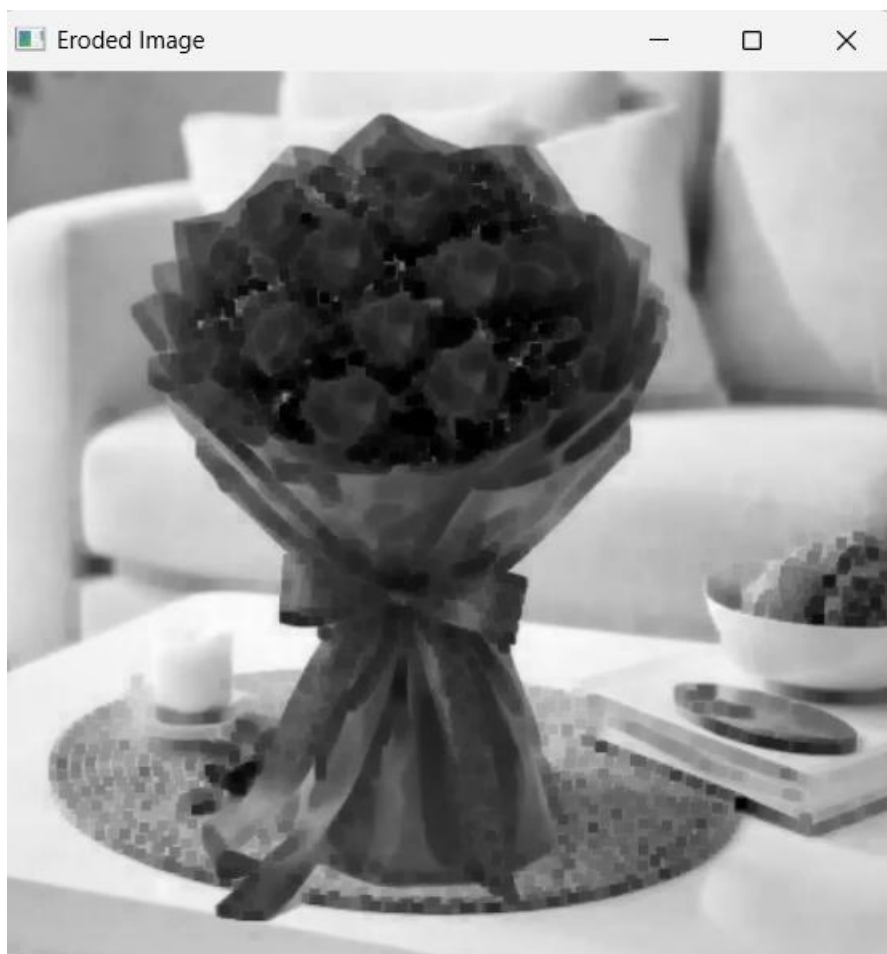
PROGRAM:

```
import cv2
import numpy as np
from tkinter import Tk, filedialog
# Hide Tkinter window
Tk().withdraw()
# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)
if not file_path:
    print("No image selected!")
    exit()
img = cv2.imread(file_path)
if img is None:
    print("Unable to load image!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
kernel = np.ones((5,5), np.uint8)
erosion = cv2.erode(gray, kernel, iterations=1)
cv2.imshow("Original Image", gray)
cv2.imshow("Eroded Image", erosion)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-20

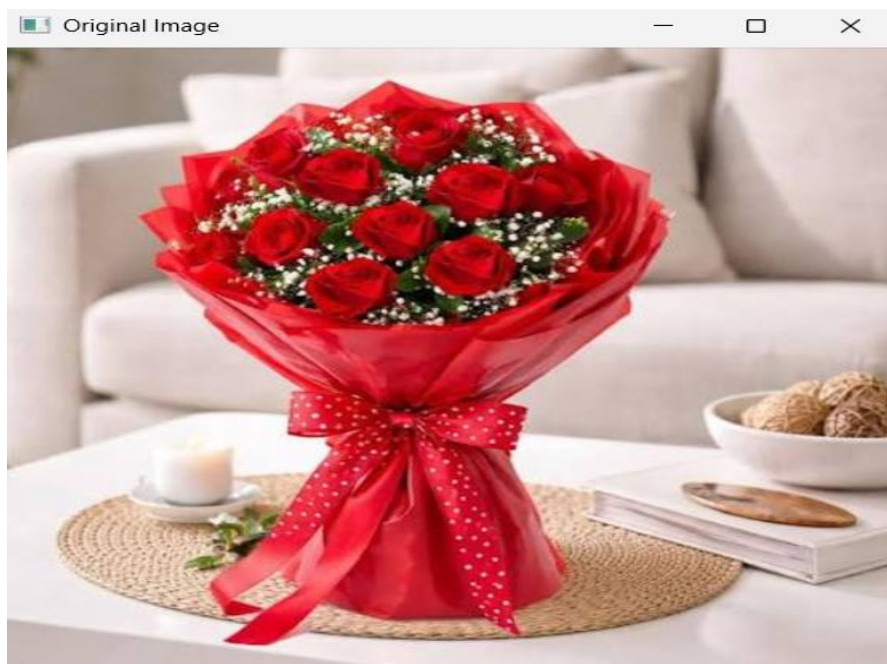
AIM:

To implement the dilation morphological operation using OpenCV.

PROGRAM:

```
import cv2
import numpy as np
from tkinter import Tk, filedialog
# Hide Tkinter window
Tk().withdraw()
# Select image
file_path = filedialog.askopenfilename(
    title="Select an Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)
if not file_path:
    print("No image selected!")
    exit()
img = cv2.imread(file_path)
if img is None:
    print("Unable to load image!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
kernel = np.ones((5,5), np.uint8)
dilation = cv2.dilate(gray, kernel, iterations=1)
cv2.imshow("Original Image", gray)
cv2.imshow("Dilated Image", dilation)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-21

AIM:

To implement the **Opening** morphological operation using OpenCV to remove small objects/noise from the foreground of an image.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

Tk().withdraw()

file_path = filedialog.askopenfilename(

    title="Select an Image",

    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]

)

if not file_path:

    print("No image selected!")

    exit()

# Read the image in color

img = cv2.imread(file_path)

if img is None:

    print("Unable to load image!")

    exit()

kernel = np.ones((5,5), np.uint8)

opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)

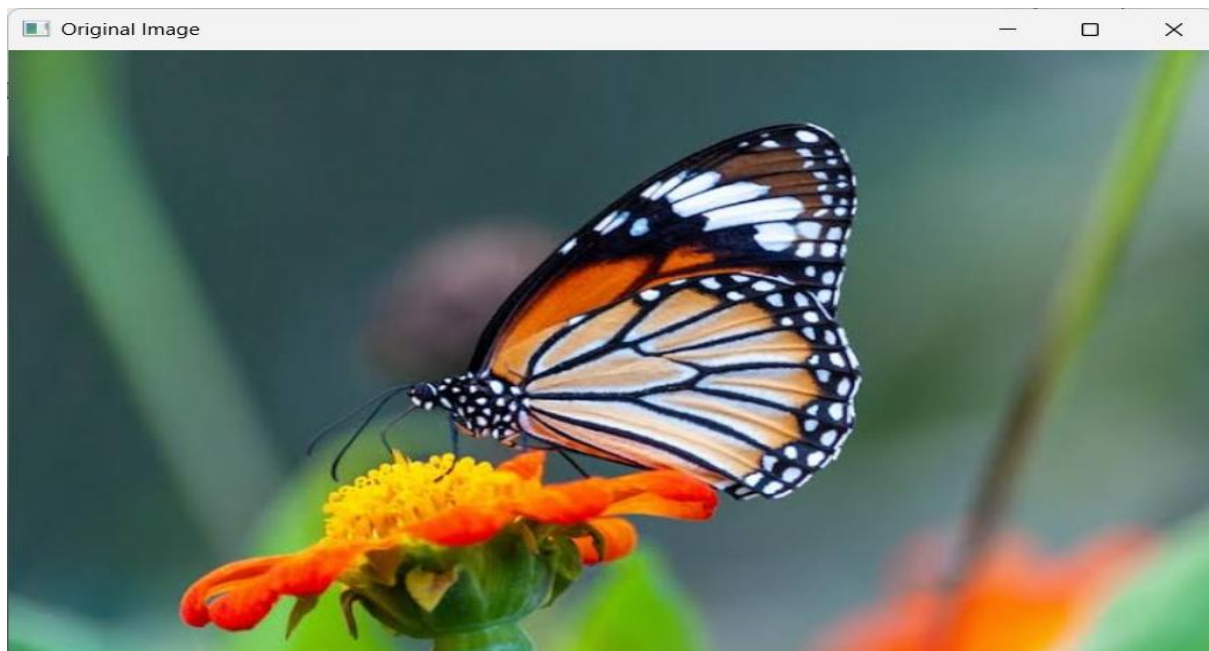
cv2.imshow("Original Image", img)
```

```
cv2.imshow("Opening Image", opening)
```

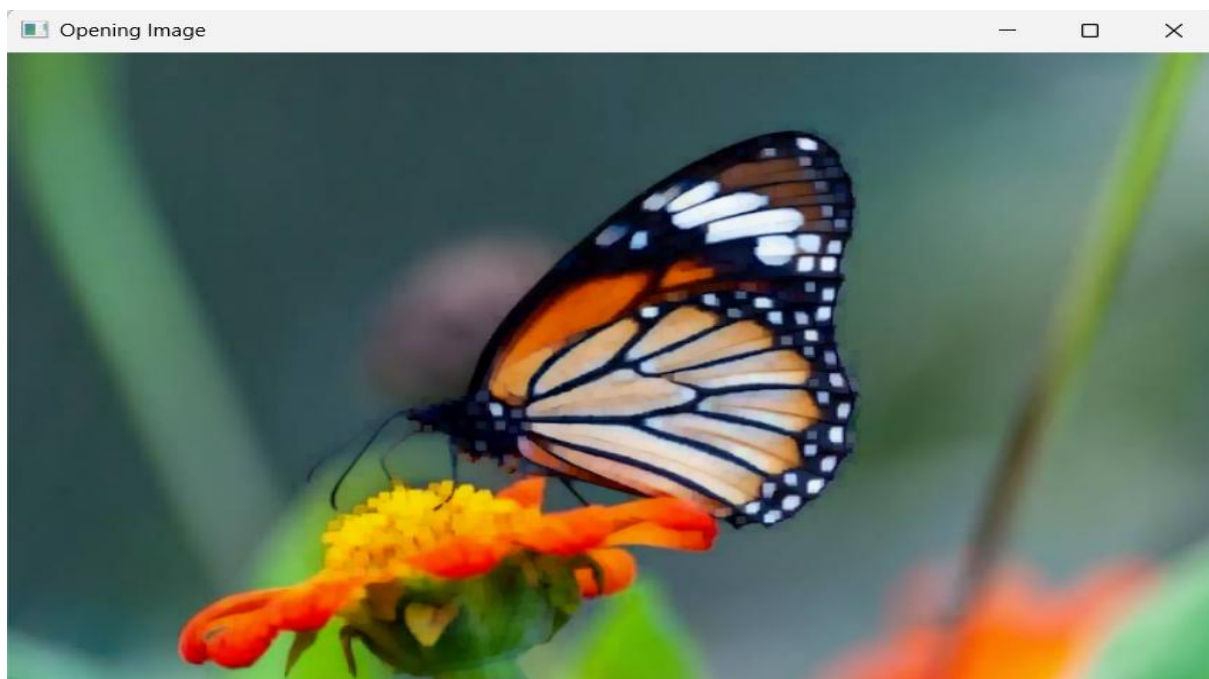
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-22

AIM:

To implement the **Closing** morphological operation using OpenCV to fill small holes in the foreground.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

Tk().withdraw()

# Select image

file_path = filedialog.askopenfilename(

    title="Select an Image",

    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]

)

if not file_path:

    print("No image selected!")

    exit()

# Read image in color

img = cv2.imread(file_path)

if img is None:

    print("Unable to load image!")

    exit()

kernel = np.ones((5,5), np.uint8)

closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
```

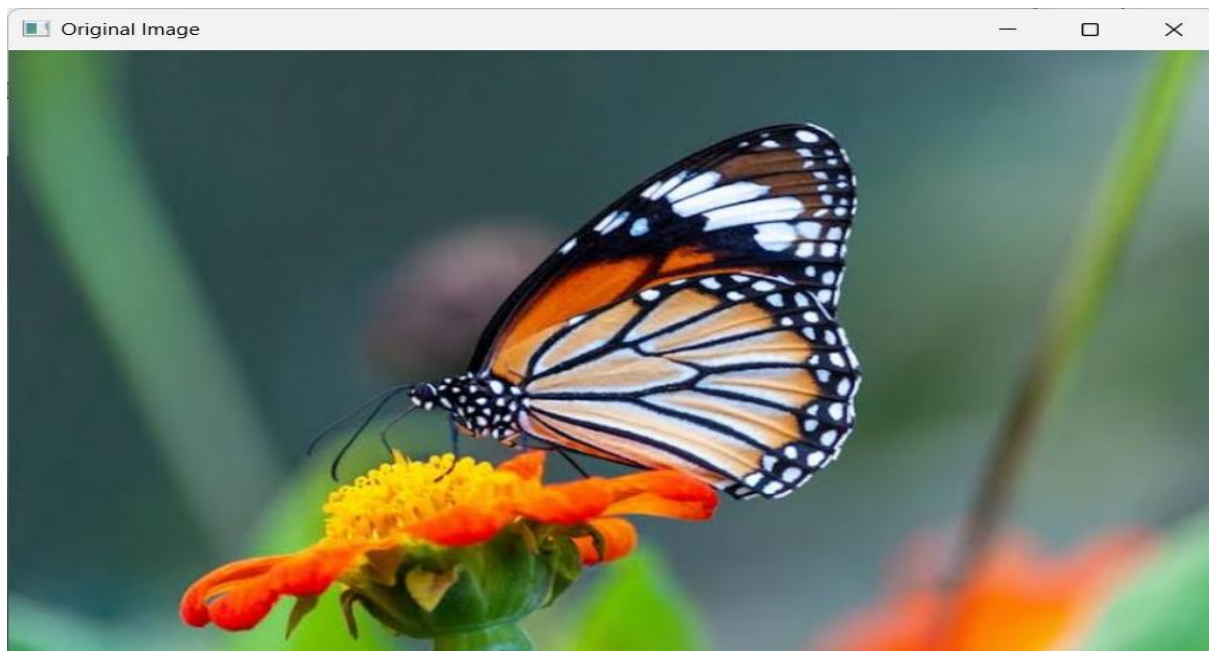
```
cv2.imshow("Original Image", img)

cv2.imshow("Closing Image", closing)

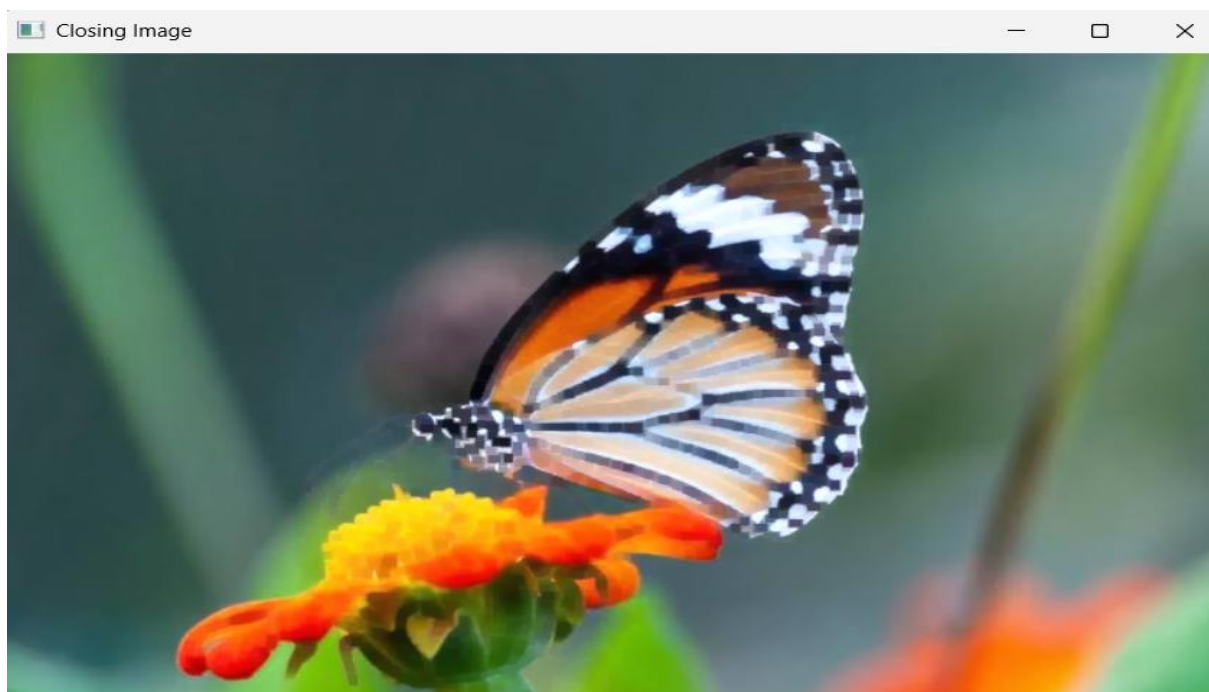
cv2.waitKey(0)

cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-23

AIM:

To implement the **Top Hat Morphological Operation** using OpenCV to extract bright objects from the background of an image.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(

    title="Select an Image",

    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]

)

if not file_path:

    print("No image selected!")

    exit()

# Read image in color
img = cv2.imread(file_path)

if img is None:

    print("Unable to load image!")

    exit()

kernel = np.ones((9,9), np.uint8)
```

```
tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
```

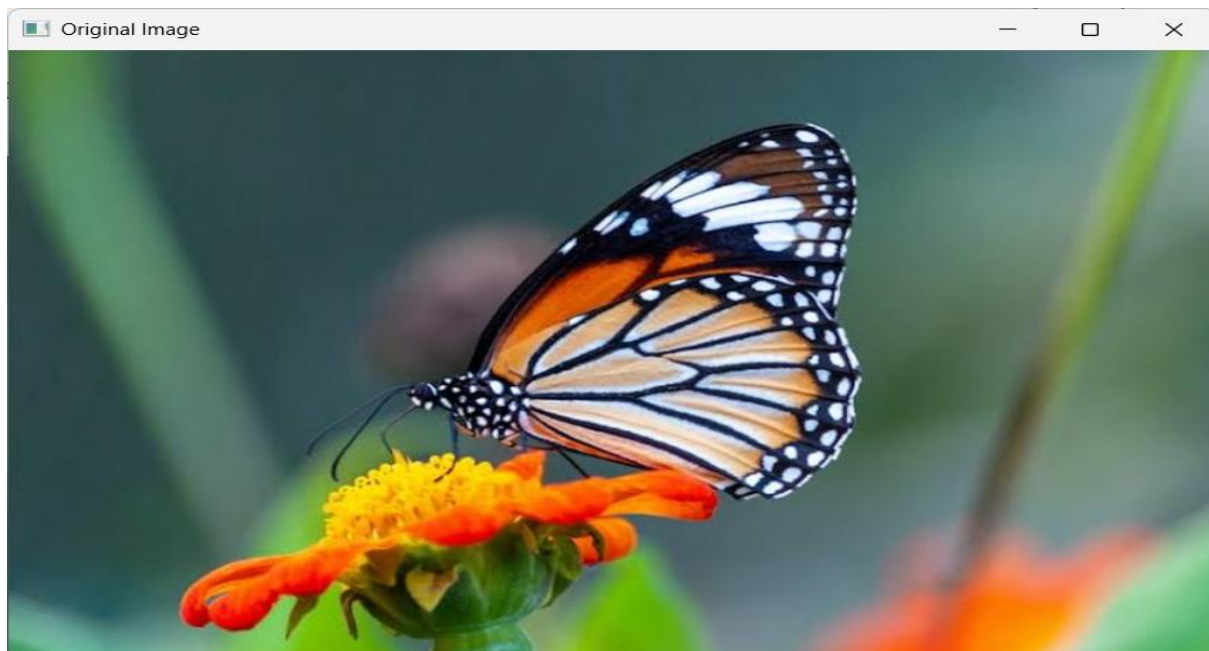
```
cv2.imshow("Original Image", img)
```

```
cv2.imshow("Top Hat Image", tophat)
```

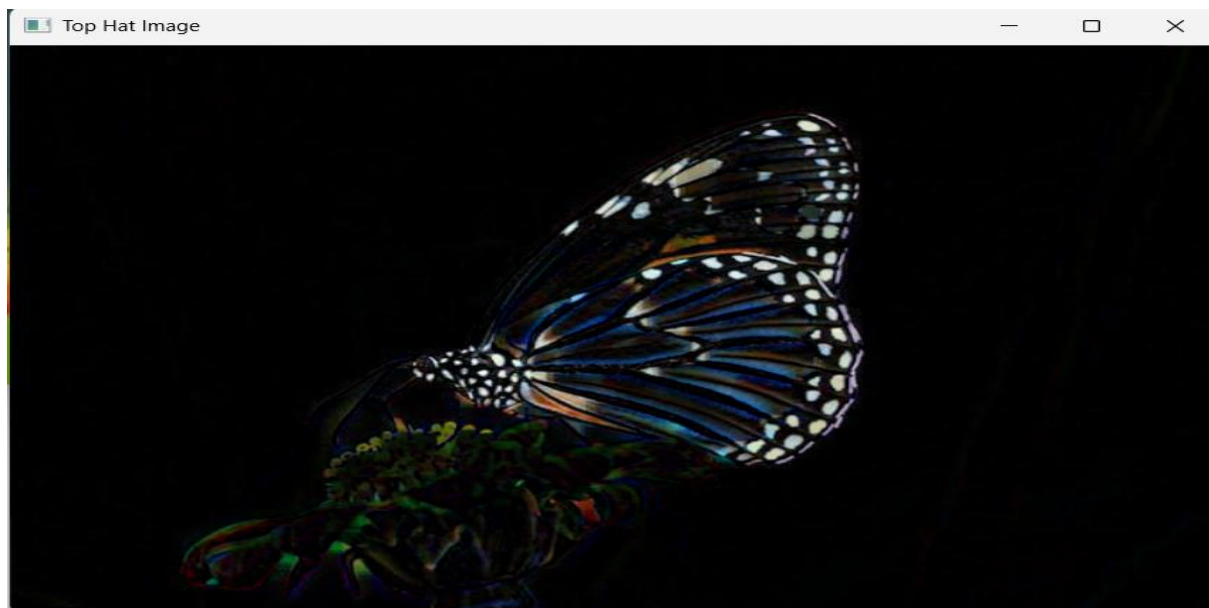
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-24

AIM:

To implement the **Black Hat Morphological Operation** using OpenCV to extract dark objects or dark regions from the background of an image.

PROGRAM:

```
import cv2

import numpy as np

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select image
file_path = filedialog.askopenfilename(

    title="Select an Image",

    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]

)

if not file_path:

    print("No image selected!")

    exit()

# Read image in color
img = cv2.imread(file_path)

if img is None:

    print("Unable to load image!")

    exit()

kernel = np.ones((9,9), np.uint8)
```

```
blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)

# Display images

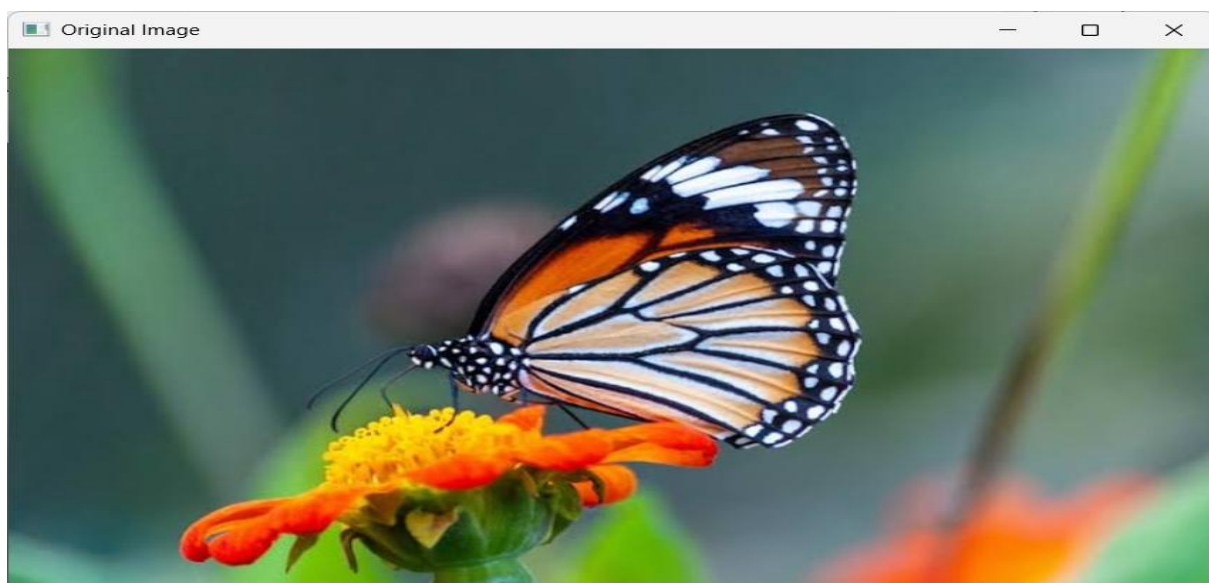
cv2.imshow("Original Image", img)

cv2.imshow("Black Hat Image", blackhat)

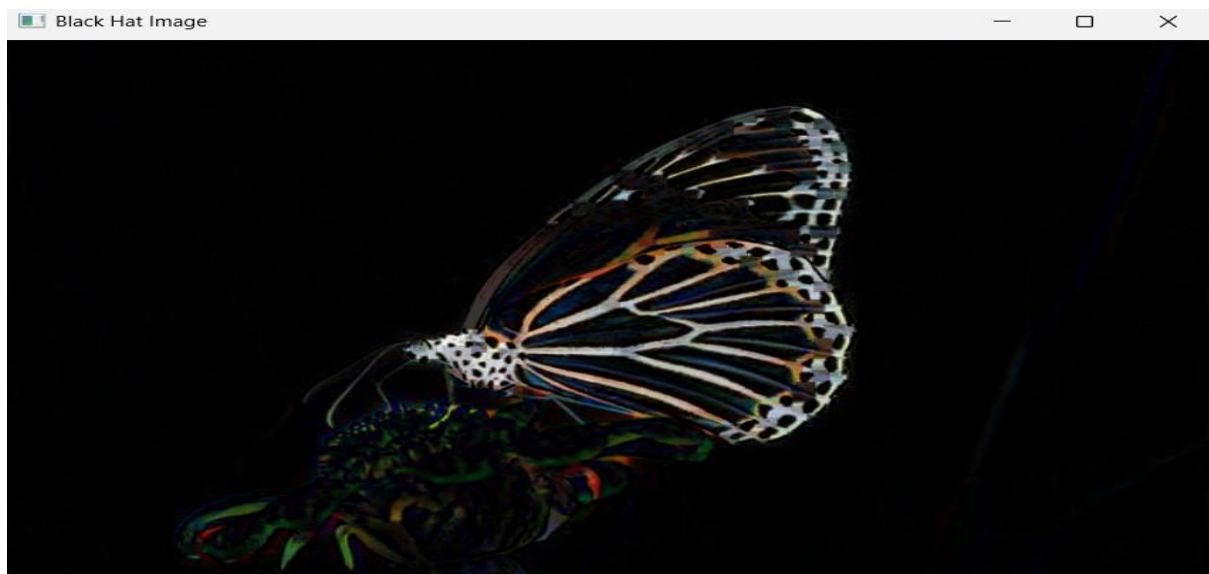
cv2.waitKey(0)

cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-25

AIM:

To recognize a **watch from a given image** using general object recognition techniques in OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select input image
file_path = filedialog.askopenfilename(
    title="Select Input Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not file_path:
    print("No input image selected!")
    exit()

# Select reference watch image
ref_path = filedialog.askopenfilename(
    title="Select Reference Watch Image",
    filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
)

if not ref_path:
    print("No reference image selected!")
```

```

    exit()

# Read images

img = cv2.imread(file_path)

ref = cv2.imread(ref_path)

if img is None or ref is None:

    print("Unable to load image!")

    exit()

# Convert to grayscale

gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray_ref = cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY)

# ORB detector

orb = cv2.ORB_create()

# Detect features

kp1, des1 = orb.detectAndCompute(gray_ref, None)

kp2, des2 = orb.detectAndCompute(gray_img, None)

if des1 is None or des2 is None:

    print("Watch not recognized!")

    exit()

if len(matches) >= 10:

    print("Watch Recognized!")

    result = cv2.drawMatches(

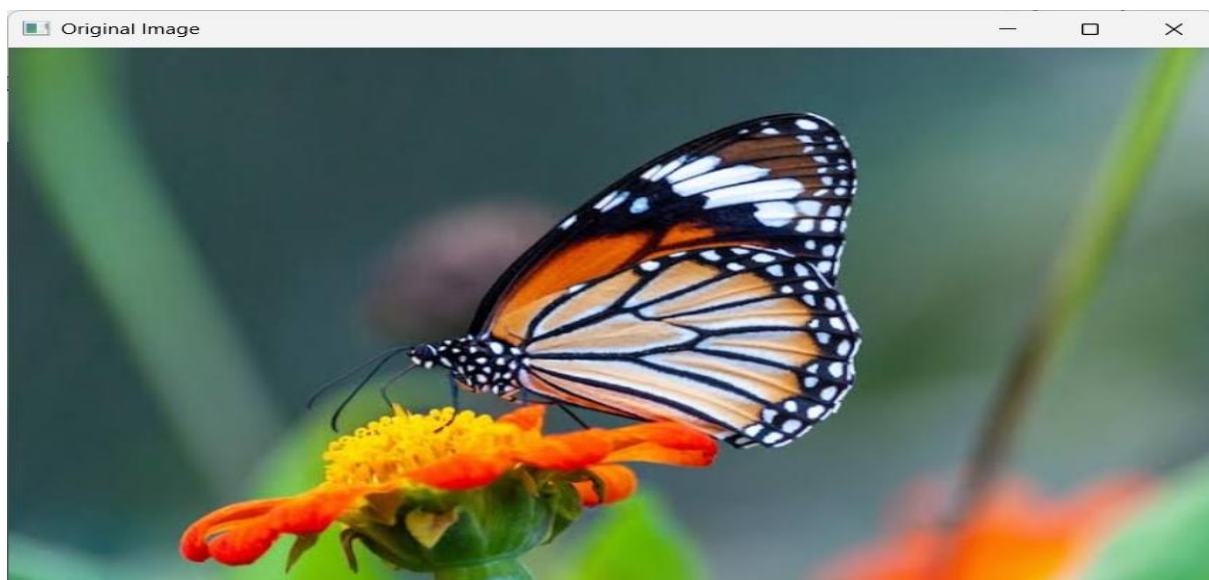
        None,

        flags=2

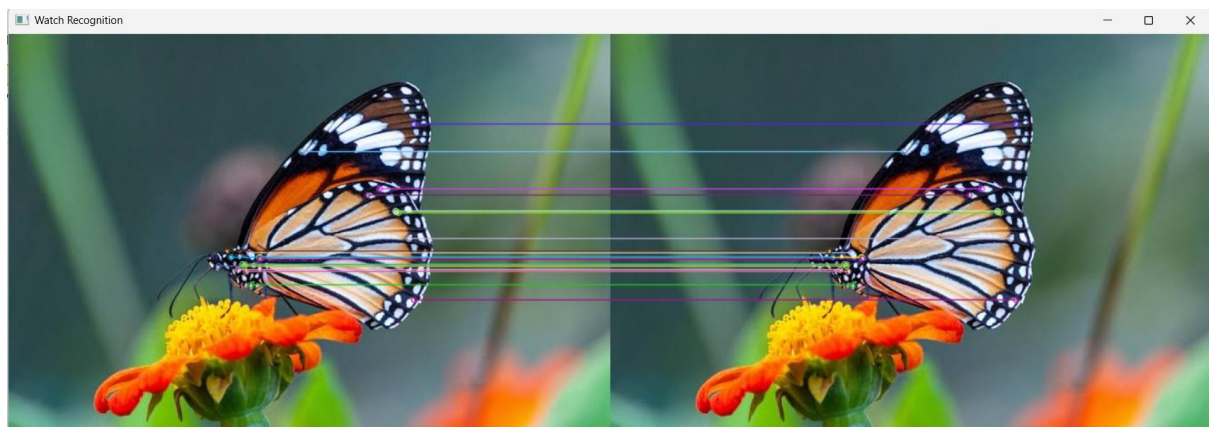
```

```
)  
  
cv2.imshow("Watch Recognition", result)  
  
else:  
  
    print("Watch Not Recognized!")  
  
    cv2.imshow("Input Image", img)  
  
cv2.waitKey(0)  
  
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-26

AIM:

To implement a function to reverse the frames of a video and create a video in reverse mode using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select video
file_path = filedialog.askopenfilename(
    title="Select a Video",
    filetypes=[("Video Files", "*.mp4 *.avi *.mov *.mkv")]
)

if not file_path:
    print("No video selected!")
    exit()

# Open video
cap = cv2.VideoCapture(file_path)

if not cap.isOpened():
    print("Unable to open video!")
    exit()

# Get video properties
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
```

```
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```
fps = cap.get(cv2.CAP_PROP_FPS)
```

```
if fps == 0:
```

```
    fps = 25
```

```
# Read frames
```

```
frames = []
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    frames.append(frame)
```

```
cap.release()
```

```
# Reverse frames
```

```
frames.reverse()
```

```
# Create output video
```

```
out = cv2.VideoWriter(
```

```
    "reverse_video.mp4",
```

```
    cv2.VideoWriter_fourcc(*"mp4v"),
```

```
    fps,
```

```
    (width, height)
```

```
)
```

```
# Write reversed frames
```

```
for frame in frames:
```

```
    out.write(frame)
```

```
out.release()

print("Reverse video created successfully: reverse_video.mp4")

# Display reversed video

for frame in frames:

    cv2.imshow("Reverse Video", frame)

    if cv2.waitKey(int(1000 / fps)) & 0xFF == 27:

        break

cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-27

AIM:

To implement a function to reverse the frames of a video and create a video in reverse mode using OpenCV.

PROGRAM:

```
import cv2

from tkinter import Tk, filedialog

# Hide Tkinter window
Tk().withdraw()

# Select video
file_path = filedialog.askopenfilename(
    title="Select a Video",
    filetypes=[("Video Files", "*.mp4 *.avi *.mov *.mkv")]
)

if not file_path:
    print("No video selected!")
    exit()

# Open video
cap = cv2.VideoCapture(file_path)

if not cap.isOpened():
    print("Unable to open video!")
    exit()

# Get video properties
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
```

```
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```
fps = cap.get(cv2.CAP_PROP_FPS)
```

```
if fps == 0:
```

```
    fps = 25
```

```
# Read frames
```

```
frames = []
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    frames.append(frame)
```

```
cap.release()
```

```
# Reverse frames
```

```
frames.reverse()
```

```
# Create output video
```

```
out = cv2.VideoWriter(
```

```
    "reverse_video.mp4",
```

```
    cv2.VideoWriter_fourcc(*"mp4v"),
```

```
    fps,
```

```
    (width, height)
```

```
)
```

```
# Write reversed frames
```

```
for frame in frames:
```

```
    out.write(frame)
```

```
out.release()

print("Reverse video created successfully: reverse_video.mp4")

# Display reversed video

for frame in frames:

    cv2.imshow("Reverse Video", frame)

    if cv2.waitKey(int(1000 / fps)) & 0xFF == 27:

        break

cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-28

AIM: To implement a vehicle detection algorithm using OpenCV to detect and locate vehicles in each frame of a video.

PROGRAM:

```
from ultralytics import YOLO

import cv2

# Load YOLO model

print("Loading YOLO model...")

model = YOLO("yolov8n.pt")

print("Model loaded successfully.")

# Open video

video_path = r"C:\Users\Haritha\Downloads\vehicles.mp4"

cap = cv2.VideoCapture(video_path)

# Check video

if not cap.isOpened():

    print("Error: Could not open video.")

    exit()

print("Video opened successfully.")

print("Press Q to stop the video.")

# Create resizable window

cv2.namedWindow("Vehicle Detection", cv2.WINDOW_NORMAL)

# Set window size

cv2.resizeWindow("Vehicle Detection", 1000, 700)

while True:

    ret, frame = cap.read()

    if not ret:

        print("Video finished.")

        break

    # Detect objects

    results = model(frame, verbose=False)
```

```
# Process detections
```

```
for result in results:
```

```
    if result.bboxes is None:
```

```
        continue
```

```
    for box in result.bboxes:
```

```
        class_id = int(box.cls[0])
```

```
        confidence = float(box.conf[0])
```

```
        if class_id in [2, 3, 5, 7]:
```

```
            x1, y1, x2, y2 = map(
```

```
                int,
```

```
                box.xyxy[0]
```

```
            )
```

```
            if class_id == 2:
```

```
                vehicle_name = "Car"
```

```
            elif class_id == 3:
```

```
                vehicle_name = "Motorcycle"
```

```
            elif class_id == 5:
```

```
                vehicle_name = "Bus"
```

```
            else:
```

```
                vehicle_name = "Truck"
```

```
            cv2.rectangle(
```

```
                frame,
```

```
                (x1, y1),
```

```
                (x2, y2),
```

```
                (0, 255, 0),
```

```
                2
```

```
            )
```

```
# Label
```

```
label = vehicle_name + " " + str(
```

```

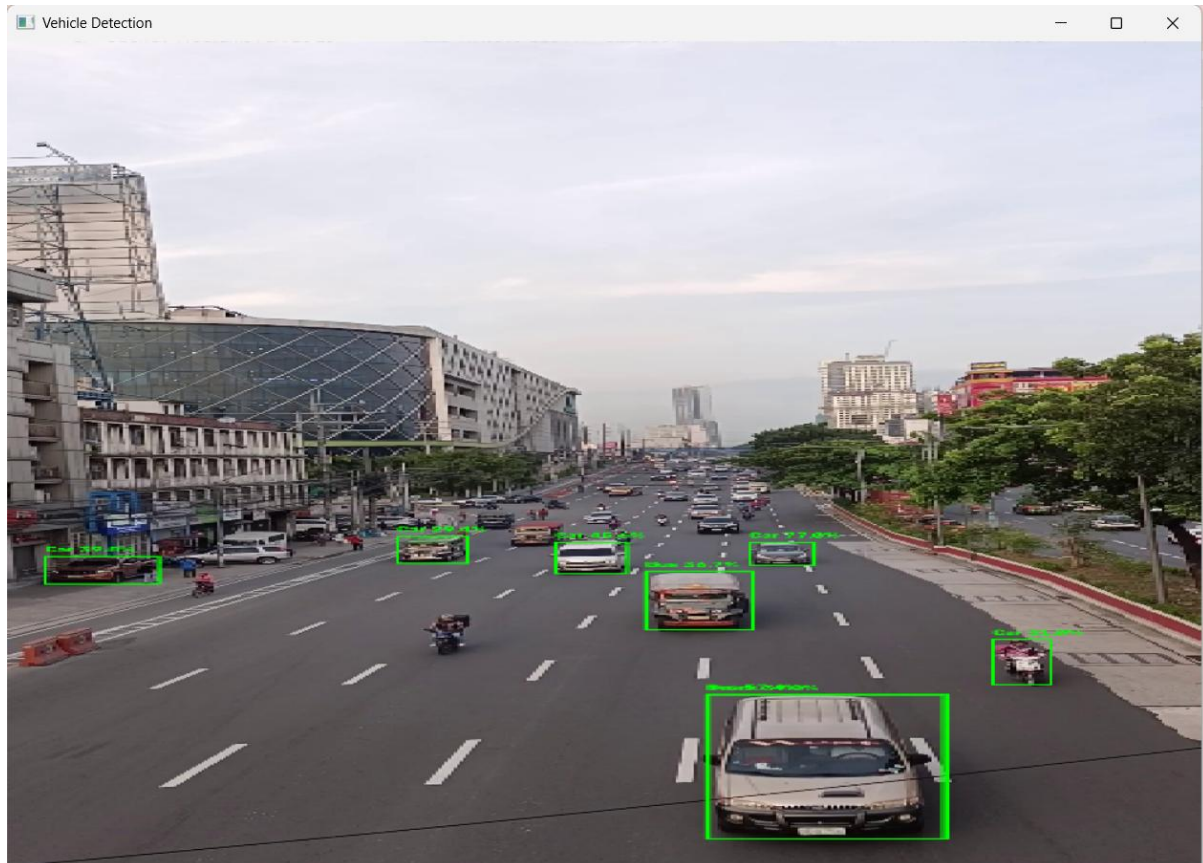
        round(confidence * 100, 1)
    ) + "%"
    cv2.putText(
        frame,
        label,
        (x1, max(y1 - 10, 20)),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 0),
        2
    )
height, width = frame.shape[:2]
max_width = 1000
max_height = 700
scale = min(
    max_width / width,
    max_height / height
)
new_width = int(width * scale)
new_height = int(height * scale)

resized_frame = cv2.resize(
    frame,
    (new_width, new_height)
)
cv2.imshow(
    "Vehicle Detection",
    resized_frame
)
if cv2.waitKey(30) & 0xFF == ord('q'):
    break

```

```
cap.release()
cv2.destroyAllWindows()
print("Vehicle detection completed.")
```

OUTPUT:



EXPERIMENT-29

AIM: To implement an eye detection algorithm using OpenCV to detect and locate human eyes in an image.

PROGRAM:

```
import cv2
eimage = cv2.imread("face.jpg")
if image is None:
```

```
print("Error: Image not found.")
exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load eye detection Haar Cascade
eye_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_eye.xml"
)

# Detect eyes
eyes = eye_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5
)

# Draw rectangle around each eye
for (x, y, w, h) in eyes:
    cv2.rectangle(
        image,
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        2
    )

# Display number of eyes
print("Number of eyes detected:", len(eyes))

# Display output
```

```
cv2.imshow("Eye Detection", image)
```

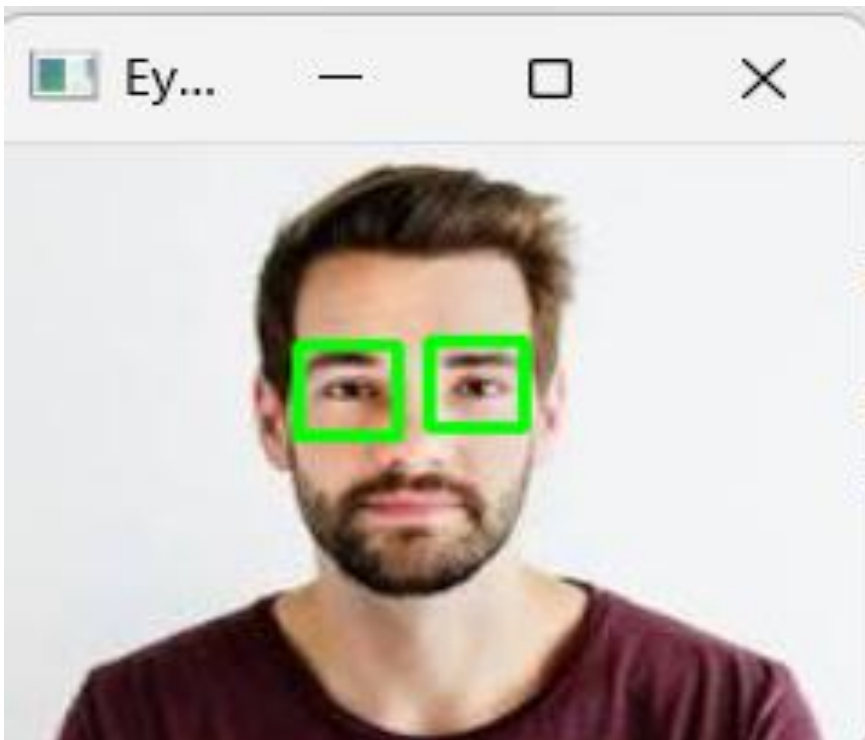
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows().
```

INPUT:



OUTPUT:



EXPERIMENT-30

AIM: To implement a smile detection algorithm using OpenCV to detect and locate a human smile in an image.

PROGRAM:

```
import cv2

image = cv2.imread("face.jpg")

if image is None:

    print("Error: Image not found.")

    exit()

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load smile detection Haar Cascade
smile_cascade = cv2.CascadeClassifier(

    cv2.data.haarcascades + "haarcascade_smile.xml"

)

smiles = smile_cascade.detectMultiScale(

    gray,

    scaleFactor=1.7,

    minNeighbors=20

)

for (x, y, w, h) in smiles:

    cv2.rectangle(

        image,

        (x, y),

        (x + w, y + h),

        (0, 0, 255),

        2

    )

print("Number of smiles detected:", len(smiles))
```

```
cv2.imshow("Smile Detection", image)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-31

AIM: To implement a segmentation algorithm using OpenCV to segment the given input image based on the given threshold value.

PROGRAM:

```
import cv2

# Read the input image
image = cv2.imread("images.jpg", 0)

# Check whether image is loaded
if image is None:
    print("Error: Image not found.")
    exit()

# Enter threshold value
threshold_value = int(input("Enter threshold value (0-255): "))

# Apply thresholding
ret, segmented = cv2.threshold(
    image,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

# Display original image
cv2.imshow("Original Image", image)

# Display segmented image
cv2.imshow("Segmented Image", segmented)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-32

AIM: To write a Python function to create a white image of a size entered by the user and create four boxes of black, blue, green, and red colors at each corner of the image. The size of each colored box should be 1/10th of the image size.

PROGRAM:

```
import cv2

import numpy as np

# Get image size from user
height = int(input("Enter image height: "))
width = int(input("Enter image width: "))

# Create a white image
image = np.ones((height, width, 3), dtype=np.uint8) * 255

# Calculate box size
box_height = height // 10
box_width = width // 10

# Top-left corner - Black
image[0:box_height, 0:box_width] = (0, 0, 0)

# Top-right corner - Blue
image[0:box_height, width-box_width:width] = (255, 0, 0)

# Bottom-left corner - Green
image[height-box_height:height, 0:box_width] = (0, 255, 0)

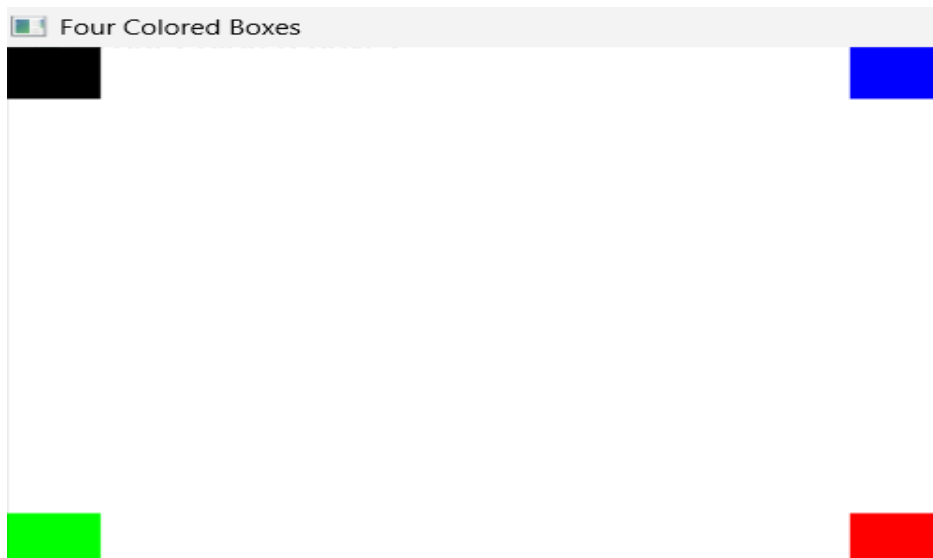
# Bottom-right corner - Red
image[height-box_height:height, width-box_width:width] = (0, 0, 255)

# Display image
cv2.imshow("Four Colored Boxes", image)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT:



EXPERIMENT-33

AIM: To write a Python function to create a white image of a size entered by the user and create a rectangle using OpenCV.

PROGRAM:

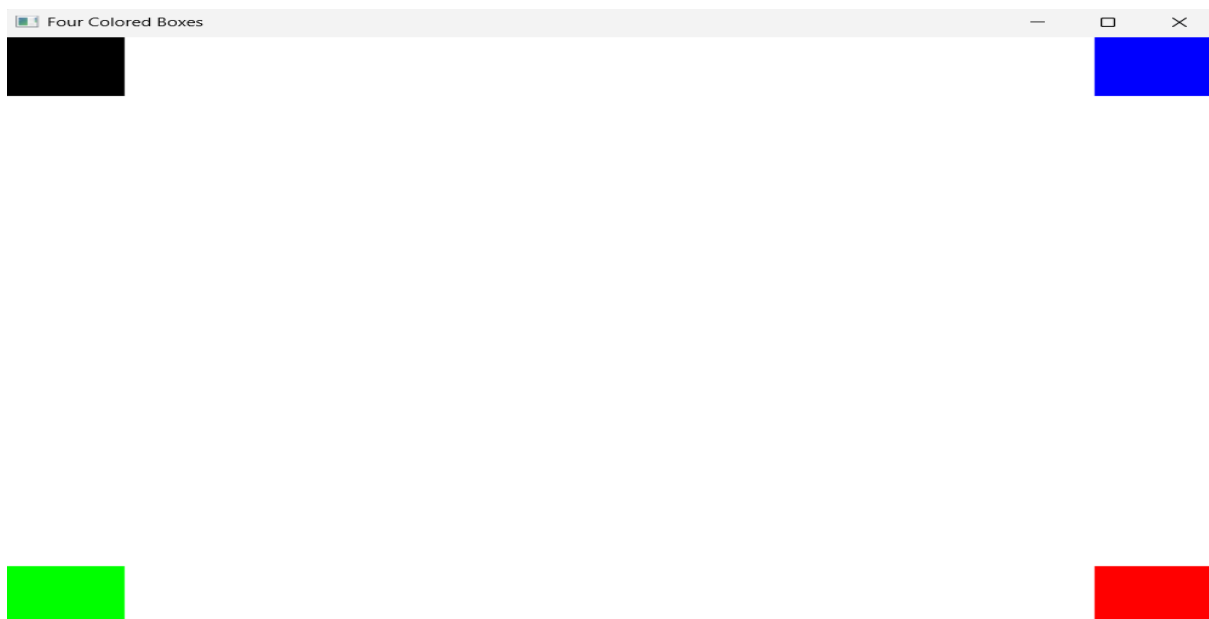
```
import cv2
import numpy as np
# Get image size from user
height = int(input("Enter image height: "))
width = int(input("Enter image width: "))
# Create a white image
image = np.ones((height, width, 3), dtype=np.uint8) * 255
# Draw rectangle
cv2.rectangle(
    image,
    (width // 4, height // 4),
    (3 * width // 4, 3 * height // 4),
    (0, 0, 255),
```

```

3
)
# Display image
cv2.imshow("Rectangle", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

OUTPUT:



EXPERIMENT-34

AIM: To write a Python function to create a white image of a size entered by the user and create a circle using OpenCV.

PROGRAM:

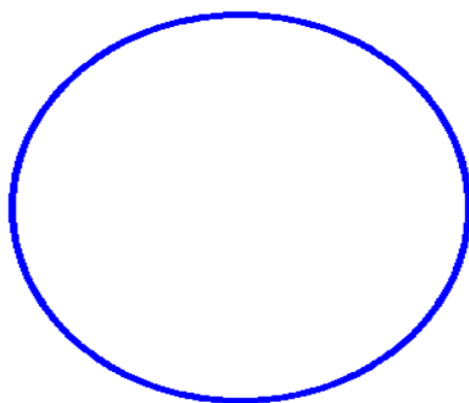
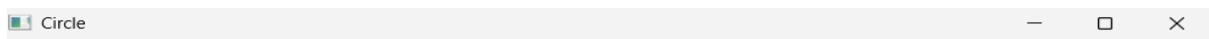
```

import cv2
import numpy as np
# Get image size from user
height = int(input("Enter image height: "))
width = int(input("Enter image width: "))
# Create a white image
image = np.ones((height, width, 3), dtype=np.uint8) * 25

```

```
# Calculate center and radius
center = (width // 2, height // 2)
radius = min(width, height) // 4
# Draw circle
cv2.circle(
    image,
    center,
    radius,
    (255, 0, 0),
    3
)
# Display the image
cv2.imshow("Circle", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



EXPERIMENT-35

AIM: To write a Python function to create a text string entered by the user and display the text on an image using OpenCV.

PROGRAM:

```
import cv2

import numpy as np

# Get text from user
text = input("Enter the text: ")

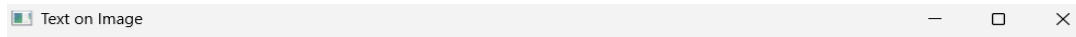
# Create a white image
image = np.ones((500, 800, 3), dtype=np.uint8) * 255

# Display text on image
cv2.putText(
    image,
    text,
    (100, 250),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.5,
    (0, 0, 0),
    3
)

# Display the image
cv2.imshow("Text on Image", image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



Welcome to OpenCV

EXPERIMENT-36

AIM: To write a Python function to subtract the background of the given input image based on color levels using OpenCV.

PROGRAM:

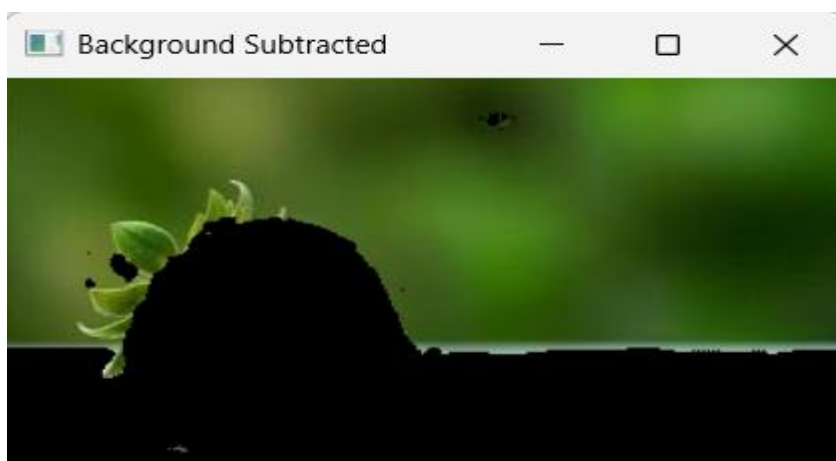
```
import cv2
import numpy as np
# Read the input image
image = cv2.imread("input.jpg")
# Check whether image is loaded
if image is None:
    print("Error: Image not found.")
    exit()
# Convert image from BGR to HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
# Define color range for background
lower = np.array([35, 40, 40])
upper = np.array([85, 255, 255])
```

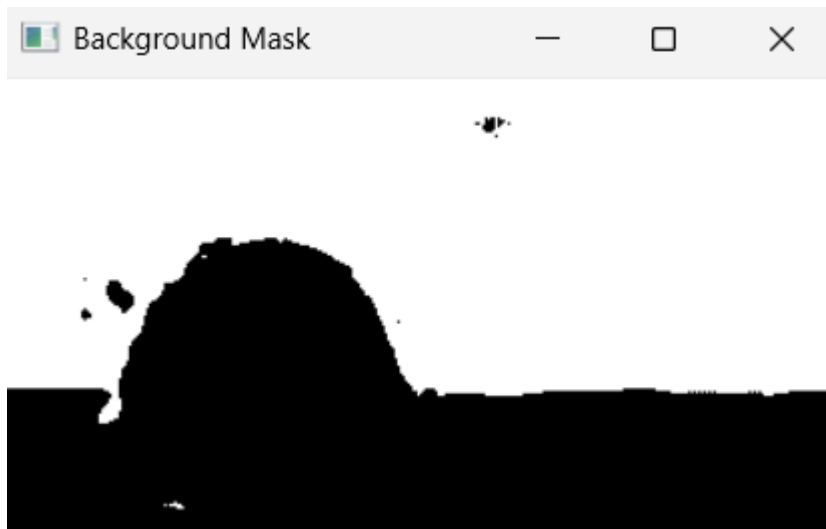
```
# Create mask for the background color
mask = cv2.inRange(hsv, lower, upper)
# Subtract background
result = cv2.bitwise_and(image, image, mask=mask)
# Display original image
cv2.imshow("Original Image", image)
# Display background mask
cv2.imshow("Background Mask", mask)
# Display result
cv2.imshow("Background Subtracted", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:





EXPERIMENT-37

AIM: To write a Python function to subtract the foreground of the given input image based on color levels using OpenCV.

PROGRAM:

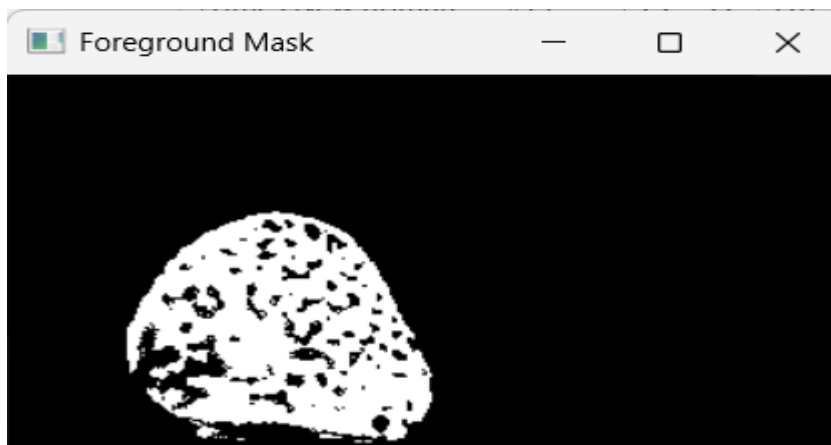
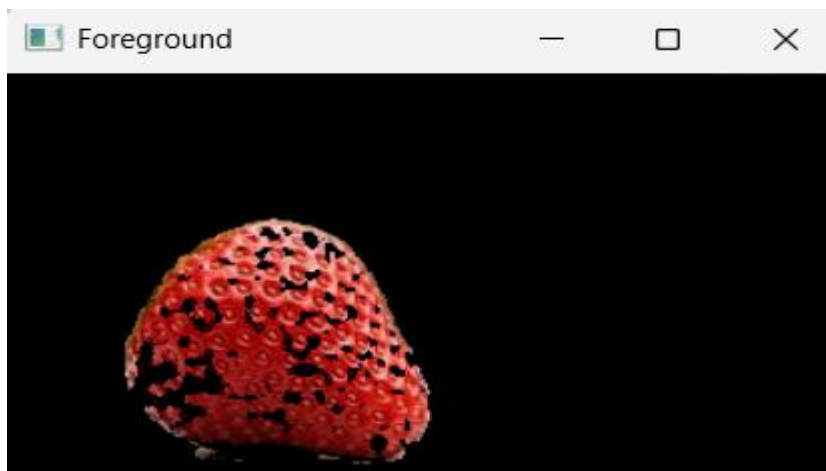
```
import cv2
import numpy as np
# Read the input image
image = cv2.imread("input.jpg")
# Check whether image is loaded
if image is None:
    print("Error: Image not found.")
    exit()
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower = np.array([0, 50, 50])
upper = np.array([20, 255, 255])
mask = cv2.inRange(hsv, lower, upper)
foreground = cv2.bitwise_and(
    image,
    image,
    mask=mask)
```

```
cv2.imshow("Original Image", image)
cv2.imshow("Foreground Mask", mask)
cv2.imshow("Foreground", foreground)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



EXPERIMENT-38

AIM: To write a Python function to count the number of faces in the given input image using OpenCV.

PROGRAM:

```
import cv2

# Read the input image
image = cv2.imread("group.jpg")

# Check whether image is loaded
if image is None:
    print("Error: Image not found.")
    exit()

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades +
    "haarcascade_frontalface_default.xml"
)

# Detect faces
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5
)
```

```

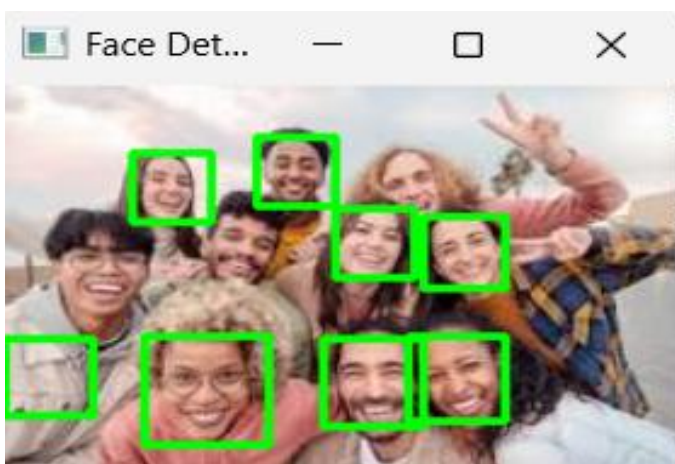
for (x, y, w, h) in faces:
    cv2.rectangle(
        image,
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        2
    )
print("Number of faces detected:", len(faces))
# Display the result
cv2.imshow("Face Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

INPUT:



OUTPUT:



EXPERIMENT-39

AIM: To write a Python function to play the given video in reverse mode in slow motion.

PROGRAM:

```
import cv2

video = cv2.VideoCapture("reverse_video.mp4")

if not video.isOpened():
    print("Error: Video not found.")
    exit()

frames = []

while True:
    ret, frame = video.read()
    if not ret:
        break
    frames.append(frame)

video.release()

print("Total frames:", len(frames))
print("Playing video in reverse slow motion...")
print("Press Q to stop.")

# Play frames in reverse order
for frame in reversed(frames):
    cv2.imshow("Reverse Slow Motion", frame)
    # 100 milliseconds delay for slow motion
    if cv2.waitKey(100) & 0xFF == ord('q'):
        break

cv2.destroyAllWindows()

print("Video completed.")
```

OUTPUT:



EXPERIMENT-40

AIM: To write a Python function to extract text from the given video using OpenCV and OCR.

PROGRAM:

```
import cv2
import pytesseract
video = cv2.VideoCapture("reverse_video.mp4")
if not video.isOpened():
    print("Error: Video not found.")
    exit()
print("Extracting text from video...")
print("Press Q to stop.")
while True:
    ret, frame = video.read()
    if not ret:
        break
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
text = pytesseract.image_to_string(gray)
if text.strip():
    print("Extracted Text:")
    print(text)
cv2.imshow("Text Extraction from Video", frame)
# Press Q to stop
if cv2.waitKey(30) & 0xFF == ord('q'):
    break
video.release()
cv2.destroyAllWindows()
print("Text extraction completed.")
```

OUTPUT:

